Classification of Human-Written and AI-Generated Text

Consolidated Project Report — Baseline Pipeline, Mixture-of-Experts Upgrade, Research Papers and Reproducibility Guide

Pravesh Gedon Sharma (B240019) & Ankit Rawat (B320032)

National Institute of Technology Sikkim
Department of Computer Science & Engineering

Course Project 2026 · Report compiled November 2026

Compiled from the project's verified metrics, result CSVs and figures — see Figure_Inventory.md and Quality_Audit.md for provenance.

Abstract

Large language models make fluent essay writing freely available, and the ability to separate human-authored text from machine-generated text now underpins academic integrity, journalism and content moderation. This report consolidates the complete course project: a three-phase comparative study of human vs AI text classification, the short-text failure it exposed, and the Mixture-of-Experts (MoE) system built to fix it. The Phase-1 baseline fuses word TF-IDF (1–2 grams, 25,000 features) with character TF-IDF (char_wb 3–5 grams, 50,000 features) into a 75,000-dimensional sparse representation and classifies with Multinomial Naive Bayes, Logistic Regression and a calibrated Linear SVM. On a balanced 20,000-essay experiment the best model reaches 99.53% validation accuracy, and 99.58% accuracy with ROC-AUC 0.9995 on a 5,000-essay held-out test. That headline number conceals a structural failure: truncating the same held-out essays to 25 words drops accuracy to 67.4%, and one sentence to 59.8%, because human-written short text is misclassified as AI in 51% of cases (human-only accuracy 48.9%) while AI recall stays at 98.7% — a one-sided collapse caused by training data with a median length of 356 words and by cross-domain shift (48.6% accuracy on unseen HC3 chat answers, worse than chance). The project therefore downloads genuine small-text data (80,811 HC3 human/ChatGPT question-answer pairs, median 117 words), 17,997 modern-2024-LLM essays (GPT-4, Claude 3, Llama 3, Command R+), augments the original corpus with truncated copies, and trains a Mixture-of-Experts: a deterministic length gate (≤150 / 151–400 / >400 words) routes to three dedicated TF-IDF + Logistic Regression experts, a learned type gate classifies chat / casual / essay from 16 stylometric features (80.5% validation), and the decision threshold adapts linearly with length (0.32 at 25 words to 0.64 at 150 words). The routed system lifts 25-word accuracy from 67.4% to 86.1% and one-sentence accuracy from 59.8% to 81.6% while holding 98.96% on full essays, and generalises to unseen domains at 98.5% (HC3 chat) and 95.7% (modern essays). The complete stack — training script, six inference front-ends (CLI, batch, REST API, LangChain tool, Streamlit application, Vercel serverless API), trained models, metrics and figures — is released for full reproducibility.

Keywords: AI-generated text detection; authorship attribution; mixture of experts; TF-IDF; stylometry; short-text classification; HC3; threshold calibration; academic integrity.

List of Abbreviations

Abbreviations used in this report.

Abbreviation	Expansion
AUC	Area Under the ROC Curve
CSV	Comma-Separated Values
F1	Harmonic mean of precision and recall
FN / FP	False Negative / False Positive
Fog	Gunning Fog readability index
FRE	Flesch Reading Ease
FKGL	Flesch–Kincaid Grade Level
HC3	Human-ChatGPT Comparison Corpus (Guo et al., 2023)
LLM	Large Language Model
LR	Logistic Regression
MoE	Mixture of Experts
NB	Multinomial Naive Bayes
NFKC	Unicode Normalization Form Compatibility Composition
OCR	Optical Character Recognition
P(AI)	Predicted probability that a text is AI-generated
PR	Precision / Recall
ROC	Receiver Operating Characteristic
SBERT	Sentence-BERT (sentence embeddings)
SVM	Support Vector Machine (linear kernel here)
TF–IDF	Term Frequency–Inverse Document Frequency
TN / TP	True Negative / True Positive
TTR	Type-Token Ratio (lexical diversity)
w	words (unit of text length)

Contents

Abstract . . . 2
List of Abbreviations . . . 3
Contents . . . 4
List of Figures . . . 9
List of Tables . . . 10

Part I — Complete Project Report 11

Chapter 1 — Introduction and Problem Statement . . . 12
1.1 Background and motivation . . . 12
1.2 Problem statement . . . 12
1.3 The length blind spot . . . 12
1.4 Contributions . . . 12
1.5 Organisation of the report . . . 13
Chapter 2 — Objectives and Scope . . . 14
2.1 Objectives . . . 14
2.2 Scope and implementation status . . . 14
2.3 Explicit non-claims . . . 14
Chapter 3 — Dataset Description and Analysis . . . 16
3.1 Corpora and scale . . . 16
3.2 Length profile . . . 16
3.3 Text characteristics . . . 17
3.4 Small-text and modern-generator corpora (MoE upgrade) . . . 17
3.5 Data availability and governance . . . 17
Chapter 4 — Baseline Methodology . . . 19
4.1 Preprocessing . . . 19
4.2 Stylometric and readability features . . . 19
4.3 Dual TF-IDF representation . . . 19
4.4 Classifiers . . . 20
4.5 Why character n-grams matter . . . 20
Chapter 5 — Experimental Setup . . . 21
5.1 Environment and protocol . . . 21
5.2 Sampling, splits and seeds . . . 21
5.3 Metrics . . . 22
5.4 Leakage controls . . . 22

Chapter 6 — Baseline Results and Evaluation . . . 23
6.1 Validation results (4,000 essays) . . . 23
6.2 Held-out test results (5,000 essays) . . . 23
6.3 Discussion . . . 24
Chapter 7 — Short-Text Failure Analysis . . . 25
7.1 Accuracy collapses monotonically with length . . . 25
7.2 The failure is one-sided: short humans become AI . . . 26
7.3 Cross-domain generalisation . . . 26
7.4 Why the model fails . . . 27
Chapter 8 — Mixture-of-Experts Architecture . . . 28
8.1 Motivation and design constraints . . . 28
8.2 Datasets for the upgrade . . . 28
8.3 Length router . . . 29
8.4 The three experts . . . 29
8.5 Type gate: chat, casual or essay . . . 29
8.6 Length-adaptive threshold calibration . . . 30
8.7 Inference path . . . 31
Chapter 9 — Mixture-of-Experts Results and Comparative Analysis . . . 32
9.1 Per-expert validation . . . 32
9.2 Routed system versus baseline . . . 32
9.3 The error structure flips . . . 33
9.4 Threshold calibration on the evaluation half . . . 34
9.5 Summary of findings . . . 34
Chapter 10 — Application and Implementation . . . 35
10.1 Streamlit demonstration interface . . . 35
10.2 Model inference engine . . . 35
10.3 Command-line interface . . . 36
10.4 REST API and LangChain integration . . . 36
10.5 System architecture . . . 37
10.6 Deployments . . . 38
Chapter 11 — Limitations and Future Work . . . 39
11.1 Limitations . . . 39
11.2 Future work . . . 39
11.3 Ethics of deployment . . . 39
Chapter 12 — Conclusion . . . 40
References (Part I) . . . 40

Part II — Research Paper **41**

Length-Expertized Authorship Detection: A Mixture-of-Experts for Human vs AI-Generated Text . 42

Abstract . . . 42

1 Introduction . . . 42

2 Related Work . . . 43

3 Datasets . . . 43

4 Methodology . . . 44

5 Experiments . . . 45

6 Results . . . 45

7 Discussion and Limitations . . . 48

8 Conclusion . . . 49

References . . . 49

Appendices . . . 49

Part III — Deep-Dive Research Paper **50**

Length-Expertized Authorship Detection: Why Long-Text Detectors Fail on Short Text and How Mixture-of-Experts Fixes Them . . . 51

Abstract . . . 51

1 Introduction . . . 51

1.1 The length blind spot in authorship detection . . . 51

1.2 Problem statement . . . 51

1.3 Contributions . . . 52

2 Related Work . . . 52

2.1 Supervised AI-text detection . . . 52

2.2 Stylometry . . . 52

2.3 Mixture of experts . . . 52

3 Datasets . . . 52

3.1 Original essays . . . 52

3.2 HC3 — genuine short chat (new) . . . 52

3.3 Modern essays — 2024-LLM audit (new) . . . 53

3.4 Truncation augmentation . . . 53

4 Problem Formulation and Method . . . 53

4.1 Notation and objective . . . 53

4.2 Text representation . . . 53

4.3 Baseline and length ablation (motivation) . . . 53

4.4 MoE architecture . . . 54

4.5 Type gate . . . 54
4.6 Calibration . . . 55
5 Experimental Setup . . . 55
5.1 Environment and protocol . . . 55
5.2 Metrics . . . 55
6 Main Results . . . 55
6.1 Per-expert and routed . . . 55
6.2 Calibration (eval half) . . . 57
7 Ablations and Analysis . . . 57
7.1 Length ablation — the flip in detail . . . 57
7.2 Sentence ablation . . . 58
7.3 Type analysis . . . 58
7.4 What did not help . . . 58
8 Limitations and Future Work . . . 59
9 Conclusion . . . 59
References . . . 59
Appendices . . . 60
A Hyperparameters . . . 60
B Reproducibility checklist . . . 60
C Full routed table . . . 60

Part IV — Code Walkthrough & Reproducibility Guide 61

1 Overview and Architecture . . . 62
2 Repository Map (file by file) . . . 63
3 src/ — Shared Utilities . . . 65
3.1 preprocessing.py . . . 65
3.2 stylometrics.py . . . 65
3.3 evaluate.py and semantic.py . . . 65
4 Training Workflow (train_baseline.py — 11 steps, 33 s) . . . 65
5 Inference Workflow (test/inference.py) . . . 66
6 Installation and Environment . . . 67
7 How to Run — The Six Front-ends and Copy-Paste Commands . . . 67
7.1 Train and evaluate . . . 68
7.2 Inference — all six ways . . . 68
7.3 Batch evaluation and reports . . . 69
8 Outputs — What Each File Means . . . 69

9 Troubleshooting and FAQ 69
10 Reproducibility Checklist 70
Appendix A — Key Snippet: load → vectorize → predict 71
Appendix B — Deployment 71

List of Figures

Figure 1: Class balance of the corpora: train.csv is perfectly balanced (145,150 per class); test.csv is human-heavy (62.8% human). 16

Figure 2: Essay length distribution (characters, 5,000-sample test set). The corpus contains essentially no natively short texts. 17

Figure 3: Dual TF-IDF feature fusion and the three linear classifiers, as implemented in train_baseline.py. 20

Figure 4: Training workflow of train_baseline.py — eleven logged steps, 33 s end-to-end with the data present. . . 21

Figure 5: Validation accuracy, F1 and ROC-AUC of the three baseline classifiers (4,000-sample validation split). . 23

Figure 6: Confusion matrices on the 5,000-sample held-out test: Logistic Regression (left) and calibrated Linear SVM (right). 24

Figure 7: ROC curves of Logistic Regression and Linear SVM on the held-out test sample; both reach AUC ≈ 0.999.[24]

Figure 8: Accuracy versus text length on the 5,000 held-out essays: the frozen baseline (orange) collapses as the input shrinks; the routed MoE (blue, developed in Chapters 8–9) lifts the short regime. 25

Figure 9: Length-conditional accuracy split by true class. Left: human-only accuracy — the baseline falls to 48.9% at 25 words. Right: AI-only accuracy — the error direction inverts under the MoE. 26

Figure 10: Mixture-of-Experts architecture: deterministic length gate, three per-bucket TF-IDF + Logistic Regression experts, learned type gate (chat / casual / essay) and length-adaptive decision threshold. 28

Figure 11: Type-gate per-class precision, recall and F1 (validation accuracy 80.5%, n = 1,500 per class). 30

Figure 12: F1 versus decision threshold for four truncation lengths. The optimum drifts right with length (25w → 0.32 … 150w → 0.64); no single global threshold serves all regimes. 30

Figure 13: Per-expert validation accuracy, F1 and ROC-AUC — every expert reaches ≥ 0.97 within its own length bucket. 32

Figure 14: Cross-domain evaluation. The baseline collapses to 48.6% on unseen HC3 chat answers (worse than chance) and to 85.5% on unseen 2024-LLM essays; the routed MoE holds 98.5% and 95.7%. 33

Figure 15: The 25-word precision/recall flip: the baseline over-calls AI (P = 0.53, R = 0.99); the MoE under-calls AI (P = 0.93, R = 0.68) while winning on total accuracy. 33

Figure 16: Screenshot of the Streamlit demonstration interface (app/app.py) performing live MoE inference with reliability warning, routed metrics and stylometric evidence. 35

Figure 17: Inference workflow: every front-end (CLI, batch, API, LangChain, Streamlit, Vercel) goes through test/inference.py. 36

Figure 18: End-to-end system architecture: local training produces the shared .pkl artifacts consumed by one inference core and six front-ends, with GitHub Pages and Vercel deployments on top. 37

List of Tables

Table 1: Implementation status of every project component (verified against the repository). . . . 14

Table 2: The four corpora: scale, class balance and length profile. . . . 16

Table 3: Complete list of splits and random seeds. . . . 21

Table 4: Baseline validation metrics on the stratified 4,000-essay split (AI is the positive class). . . . 23

Table 5: Held-out test metrics (5,000 essays, seed 42), recomputed from the frozen model artifacts. . . . 23

Table 6: Length ablation of the frozen baseline on the 5,000 held-out essays (AI is the positive class). . . . 25

Table 7: Length-conditional accuracy split by true class (human = true-negative rate, AI = true-positive rate; 5,000 held-out essays). . . . 26

Table 8: Mixture-of-Experts training pools (balanced per class, seed 42). . . . 29

Table 9: Per-expert validation on the held-out 20% of each pool (results/moe_expert_metrics.csv). . . . 32

Table 10: Baseline versus routed Mixture-of-Experts on identical inputs (results/moe_baseline_comparison.csv vs moe_routed_metrics.csv). . . . 32

Table 11: Calibration on the held-out evaluation half (2,500 essays, seed 999): default θ = 0.50 versus length-adaptive thresholds. . . . 34

Table 12: REST API endpoints implemented in test/api.py (FastAPI, port 8001). . . . 37

Table 13: The six inference front-ends and two deployments. . . . 37

Table 14: Dataset scale (Part II). Held-out n = 5,000 (seed 42) for all comparisons. . . . 43

Table 15: Baseline validation, 4,000 essays (results/baseline_ml_metrics.csv). . . . 44

Table 16: MoE per-expert validation (results/moe_expert_metrics.csv). . . . 45

Table 17: Headline deltas (results/moe_routed_metrics.csv vs moe_baseline_comparison.csv). . . . 47

Table 18: Calibration on the eval half (acc/F1). Detail in results/moe_calibrated_metrics.csv. . . . 47

Table 19: MoE training pools (seed 42). Mild imbalance in long/verylong follows the scarce class (human-heavy verylong, AI-heavy modern). . . . 53

Table 20: Per-expert validation (results/moe_expert_metrics.csv). . . . 55

Table 21: Headline deltas (results/moe_*.csv). . . . 57

Table 22: Length-conditional accuracy split (human TN rate vs AI TP rate). MoE rescues humans (+47.9 pp @25w) at AI-recall cost (−30.7 pp) — net acc still wins. . . . 57

Table 23: Hyperparameters (all seeded). . . . 60

Table 24: Routed MoE, all evaluation conditions (results/moe_routed_metrics.csv). . . . 60

Table 25: Repository map — what each file does and how large it is. . . . 64

Table 26: The eleven logged steps of train_baseline.py. . . . 66

Table 27: The inference engine's public functions (test/inference.py, 158 lines). . . . 67

Table 28: The six inference front-ends (Part IV view; cf. Table 13 for the full system inventory). . . . 68

Table 29: Artifacts written by training and evaluation. . . . 69

Table 30: Deployment targets. . . . 71

Part I — Complete Project Report

Chapters 1–12: problem, data, baseline method, short-text failure, Mixture-of-Experts, results, application, limitations, conclusion

Chapter 1 — Introduction and Problem Statement

1.1 Background and motivation

Large language models (LLMs) such as ChatGPT, GPT-4 and Claude produce fluent, well-structured text on demand. What used to require an educated writer — a coherent essay, a news summary, a product review — is now available to anyone in seconds. The upside is obvious; the cost is that text itself stops being evidence of authorship. Universities need to protect academic integrity, newsrooms need to attribute bylines, and platforms need to moderate generated content at scale. Deciding whether a passage was *human-written* or *AI-generated* has therefore become a standard problem in natural language processing and information forensics.

The literature offers a familiar recipe: collect paired human/machine text, extract features, train a classifier, report accuracy. Benchmarks such as HC3 [1], Defactify [2] and M4 [3] follow it and produce headline numbers in the high 90s. This project started from exactly that recipe — a dual TF-IDF pipeline with classical linear classifiers — and reproduced the headline: **99.58% accuracy on held-out essays**. The interesting part of this project is what happened next: we attacked our own model with short inputs and watched it fail in a systematic, biased way, then rebuilt it until it did not.

1.2 Problem statement

Let x be a text, $y \in \{0, 1\}$ its true label (0 = human, 1 = AI), and $n(x)$ its length in words. We seek a classifier $f(x) \to \hat{y}$ that maximises accuracy when the length distribution at test time differs sharply from training: the training corpus has a median of 356 words (5th percentile 177 words), while real deployment inputs — chat replies, comments, reviews, one-sentence blurbs — may carry fewer than 50 words. Two failure mechanisms make this hard: (i) *evidence sparsity*, because TF-IDF counts, character n-grams and stylometric statistics all need tokens to stabilise; and (ii) *domain shift*, because short text is mostly chat and casual writing, a register an essay-trained model has never seen.

1.3 The length blind spot

Quantitatively, the problem is severe. The frozen Phase-1 baseline reaches 99.26% accuracy on full-length held-out essays, but only 67.4% when the same essays are truncated to their first 25 words, and 59.8% at a single sentence. More disturbing than the drop is its *direction*: at 25 words the model labels 51% of genuine human texts as AI (human-only accuracy 48.9%, worse than a coin flip) while still recognising 98.7% of AI texts. A detector that confidently accuses short humans is worse than an uninformative one — Figure 8 and Figure 9 quantify this failure and are analysed in Chapter 7.

1.4 Contributions

- **A reproducible baseline.** Dual word+character TF-IDF (75,000 features) with Multinomial Naive Bayes, Logistic Regression and a calibrated Linear SVM, trained in 33 seconds and evaluated on a 5,000-essay held-out test (best model: 99.58% accuracy, ROC-AUC 0.9995).
- **A length ablation study.** First-N-words and first-N-sentences truncations of the same held-out essays, with accuracy split by true class, revealing a one-sided false-positive collapse (Chapter 7).
- **Cross-domain evaluation.** Two never-trained-on holdouts — HC3 chat answers and 2024-LLM essays — showing the baseline falls to 48.6% and 85.5% (Chapter 7).
- **A Mixture-of-Experts detector.** A deterministic length gate, three length-specialised TF-IDF + LR experts trained on newly downloaded small-text data, a learned chat/casual/essay type gate, and length-adaptive thresholds (Chapters 8–9): 86.1% at 25 words, 98.96% full text, 98.5% on unseen chat.

- **A complete, reproducible system.** Six inference front-ends (CLI, batch, REST API, LangChain tool, Streamlit application, Vercel serverless API), trained models in version control, and all metrics released as CSV (Chapter 10, Part IV).

1.5 Organisation of the report

- **Part I (Chapters 1–12)** — the complete project report: problem, data, baseline methodology and results, short-text failure analysis, MoE architecture and results, the deployed application, limitations and conclusion.
- **Part II** — the conference-format research paper: *Length-Expertized Authorship Detection: A Mixture-of-Experts for Human vs AI-Generated Text*.
- **Part III** — the detailed research paper with formal notation, equations and ablation analysis: *Why Long-Text Detectors Fail on Short Text and How Mixture-of-Experts Fixes Them*.
- **Part IV** — technical documentation and reproducibility guide: repository map, training and inference workflows, commands, API and troubleshooting.

Figures and tables are numbered continuously across all four parts; each part keeps its own section numbering so the two papers remain readable as standalone documents.

Chapter 2 — Objectives and Scope

2.1 Objectives

1. Build an end-to-end classification pipeline that separates human-written from AI-generated text with classical, fully interpretable features (stylometrics + dual TF-IDF) and linear classifiers.
2. Quantify the pipeline's behaviour across the evaluation axes that matter — accuracy, precision, recall, F1, ROC-AUC, confusion matrices and feature-family comparisons.
3. Stress-test the trained model under length shift (truncation to 25–200 words and to single sentences) and under domain shift (unseen chat and unseen generators), and characterise the resulting error structure.
4. Acquire genuine small-text training data and modern-LLM essay data, and combine them with truncation augmentation of the original corpus.
5. Design and train a length-routed Mixture-of-Experts with a learned text-type gate and length-adaptive decision thresholds that fixes the short-text failure without sacrificing full-text accuracy.
6. Package everything as a usable system — training script, CLI, REST API, LangChain tool, web application and documentation — so that every reported number can be reproduced from the released artifacts.

2.2 Scope and implementation status

Accuracy of claims matters more than breadth in an engineering report. Every row of Table 1 states what actually exists in the repository and what was actually measured; components that were designed but not trained are marked as such and are not credited with any reported score.

Table 1: Implementation status of every project component (verified against the repository).

Component	Where it lives	Status
Dual TF-IDF baseline (NB, LR, calibrated SVM)	train_baseline.py, models/*.pkl	Implemented and evaluated — all Part I results
Stylometric features (TTR, burstiness, readability)	src/stylometrics.py	Implemented — used by the demo UI and by the MoE type gate
MoE experts, length gate, type gate, thresholds	moe/*.py, models/moe/*	Implemented and evaluated — Chapters 8–9
Inference engine + six front-ends	test/inference.py, test/, app/app.py	Implemented and exercised — Chapter 10, Part IV
EDA and feature notebooks 01–04	notebooks/	Implemented as notebooks — supporting analysis
Transformer fine-tuning (DistilBERT, RoBERTa)	notebooks/05_*.ipynb	Prepared but not trained in this report — no scores reported
Sentence-BERT semantic fusion	src/semantic.py	Scaffolding only — not wired into the inference path
Feature-family ablation study	notebooks/06_*.ipynb	Planned Phase-3 work — not part of reported results

2.3 Explicit non-claims

- No transformer (DistilBERT/RoBERTa) score is reported anywhere in this report, because no transformer was trained or evaluated as part of the recorded results.

- Stylometric ablation and Sentence-BERT fusion are *available code paths*, not completed experiments; they are listed as future work in Chapter 11.
- All reported metrics come from frozen artifacts in `models/` and `models/moe/` evaluated by the released scripts — none are hand-entered or extrapolated.

Chapter 3 — Dataset Description and Analysis

3.1 Corpora and scale

Four corpora are used. The original essay pair (`data/train.csv`, `data/test.csv`) carries columns `text, generated` where `generated=0` is human and `generated=1` is AI. The two additional corpora — HC3 question-answer pairs and modern-2024-LLM essays — were downloaded for the MoE upgrade. Table 2 summarises all four; every figure in it was recomputed directly from the CSV files during preparation of this report.

Table 2: The four corpora: scale, class balance and length profile.

Dataset	Rows	Human / AI	Median length	Role in the project
`data/train.csv`	290,300	145,150 / 145,150	356 words	Baseline training (20k balanced subsample)
`data/test.csv`	97,447	61,159 / 36,288	363 words	Held-out evaluation (5,000-sample)
HC3 `reddit_eli5` + `open_qa` + `finance`	80,811	56,115 / 24,696	117 words (61% ≤ 150w)	SHORT expert — genuine chat Q&A
Modern essays (GPT-4, Claude 3, Llama 3, Command R+)	17,997	3,000 / 14,997	510 words (68% > 400w)	LONG/VERYLONG experts — 2024 generators

The training corpus is exactly balanced (145,150 essays per class); the test corpus is human-heavy (62.8% human), which is the harder, more realistic direction for a deployed detector. Figure 1 shows the class balance of both files.

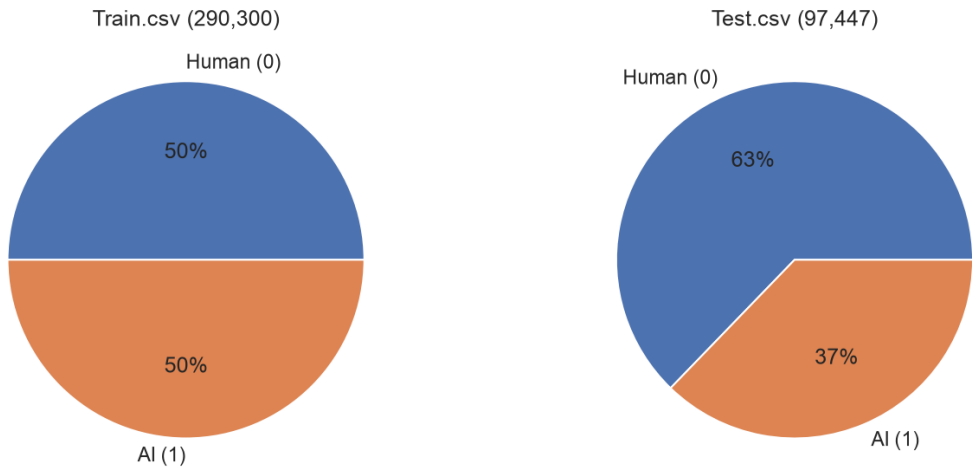


Figure 1: Class balance of the corpora: train.csv is perfectly balanced (145,150 per class); test.csv is human-heavy (62.8% human).

3.2 Length profile

The defining property of the original corpora is that they contain essays, not short text. Across all 290,300 training essays the median length is 356 words with a 5th percentile of 177 words; the test corpus mirrors this (median 363 words). In the 5,000-essay evaluation sample the mean is 391 words and the median 362, with the shortest human essay at 48 words and human essays averaging 422 words against 340 for AI essays — human writing in this corpus is simply longer. Figure 2 plots the character-length distribution: there is essentially no mass below 200 characters, which means the model never saw a short input during training. This single fact drives the failure analysed in Chapter 7.

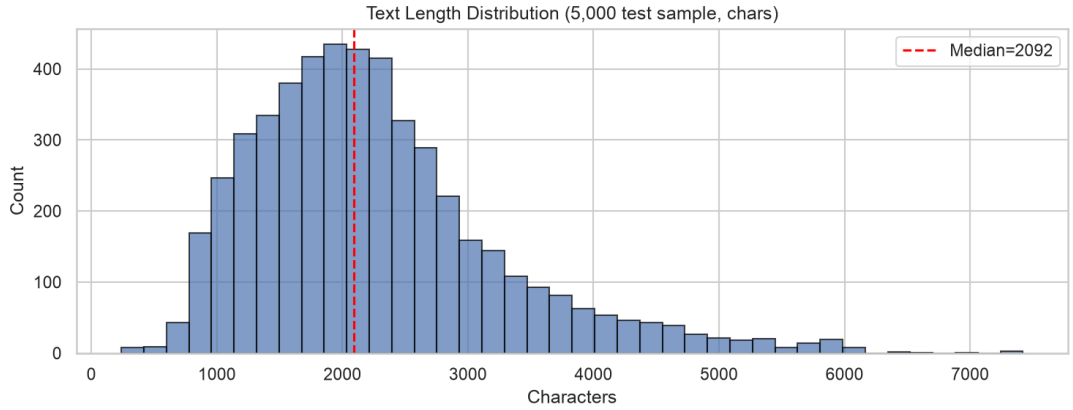


Figure 2: Essay length distribution (characters, 5,000-sample test set). The corpus contains essentially no natively short texts.

3.3 Text characteristics

Human example (verbatim from `test.csv`): *"There are many advantages TD limiting the number DF cars DN the roads…"* — OCR/typing artefacts such as `TD`, `DF`, `DN` are pervasive in human essays and are strong character n-gram cues. **AI example**: *"In today's world, having a positive attitude is a crucial factor in achieving success…"* — polished formal structure with headings such as "Pros of Distance Learning: 1. Flexibility". AI text is orthographically clean; human text is noisy. The baseline exploits exactly this asymmetry (Chapter 4, §4.5).

Because the unit of data is a multi-paragraph essay (median 363 words, roughly 2,092 characters at the sample level), chunking is required for any transformer-based alternative — one reason the classical pipeline remains the practical choice for this project.

3.4 Small-text and modern-generator corpora (MoE upgrade)

HC3 (Hello-SimpleAI/HC3, Guo et al. [1]) was flattened to one row per answer (≥ 40 characters) from three files: `reddit_eli5.jsonl`, `open_qa.jsonl` and `finance.jsonl`. It yields 80,811 paired answers (56,115 human, 24,696 ChatGPT) with a median of 117 words; 61.3% are ≤ 150 words, 34.7% fall in 151–400 words and only 4.0% exceed 400. It is the only source in the project of *genuine short chat*: the same question answered by a human and by ChatGPT.

Modern essays (`artnitolog/llm-generated-texts` [8]) supply 17,997 texts — 3,000 human essays against 14,997 machine essays from GPT-4 Turbo/Omni, Claude 3 Opus, Llama 3 70B and Command R+, which are generators the original corpus never saw. Median length 510 words, 68% longer than 400 words — this corpus audits the baseline on 2024-era models.

Truncation augmentation complements both: every essay in the 20,000-essay training subsample additionally contributes 50/100/150-word copies (prefix or random window, label preserved) to the short pool, and essays over 200 words contribute 200/300-word copies to the long pool. This teaches the short expert *short-essay* style, which HC3 cannot provide. The resulting training pools are specified in Table 8 (Chapter 8).

3.5 Data availability and governance

- `data/train.csv` (653 MB) and `data/test.csv` (222 MB) are **gitignored** — they are catalogued with Google-Drive placeholders in `data/README.md`; small tracked samples are provided as `data/sample_*.csv`.
- `data/moe/*.csv` (132 MB) is also gitignored but fully reproducible: `PYTHONPATH=. python3 moe/download_small_text.py` re-downloads both corpora from the Hugging Face Hub.

- A zipped bundle `data/ai_text_moe_data.zip` (40 MB) exists locally for offline sharing.
- All random operations are seeded (seed 42 for sampling and splits, 999 for calibration) so every table in this report can be regenerated bit-for-bit.

Chapter 4 — Baseline Methodology

The baseline answers one question with a deliberately simple model: can a linear classifier over surface statistics separate human from AI essays? Everything is implemented in `train_baseline.py` (11 logged steps) with shared helpers in `src/`.

4.1 Preprocessing

- `clean_text_for_tfidf(text)` — NFKC Unicode normalisation, whitespace collapse, lowercasing. Applied as the `preprocessor=` of the word TF-IDF so training and inference normalise text identically.
- `clean_text_for_stylometrics(text)` — preserves casing and punctuation (capital ratio, comma/semicolon rates) because stylometric features read them; it never lowercases.
- Vectorizers are **fit on the training split only**; validation, test and the MoE holdouts are merely transformed — no vocabulary or IDF leakage.

4.2 Stylometric and readability features

Although the baseline classifier consumes TF-IDF features only, the project extracts a 24-value stylometric vector (`src/stylometrics.py`) [6] for two purposes: human-readable explanations in the demo interface, and the 16 inputs of the MoE type gate. These include type-token ratio (TTR), hapax ratio, mean sentence length, burstiness (σ/μ of sentence lengths), punctuation and capitalisation rates, and three readability scores defined as:

$$FRE = 206.835 - 1.015 \cdot (N/S) - 84.6 \cdot (syll/N) \quad FKGL = 0.39 \cdot (N/S) + 11.8 \cdot (syll/N) - 15.59 \quad Fog = 0.4 \cdot [(N/S) + 100 \cdot (poly/N)] \tag{1}$$

where N is the number of words, S the number of sentences, syll/N the average syllables per word and poly/N the fraction of polysyllabic words. All three need several sentences before they stabilise — a fact that becomes decisive in Chapter 7.

4.3 Dual TF-IDF representation

Two views of every document are extracted and fused (Figure 3):

- **Word TF-IDF** — unigrams and bigrams, `ngram_range=(1,2)`, `max_features=25000`, `min_df=2`, `sublinear_tf=True`. Captures lexical choice and common phrases.
- **Character TF-IDF** — `analyzer='char_wb'`, `ngram_range=(3,5)`, `max_features=50000`, `min_df=3`. Captures typos, OCR noise, punctuation habits and sub-word morphology.
- **Fusion** — `hstack([word, char]).tocsr()` producing a **75,000-dimensional** sparse vector; the 16,000-essay training matrix holds 34.8 million non-zero cells.

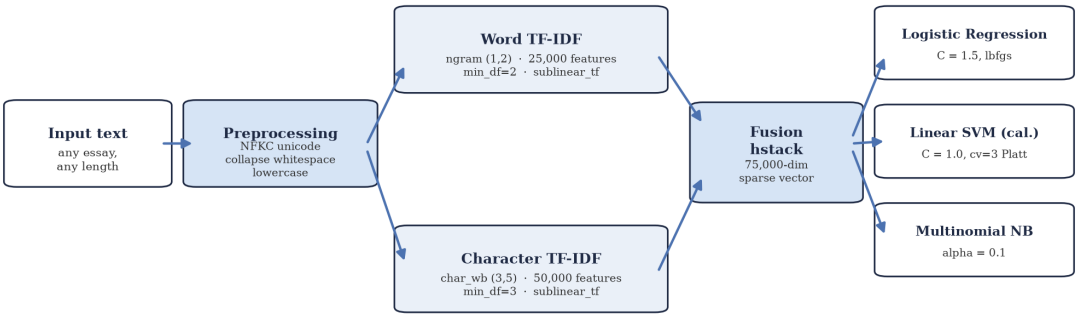


Figure 3: Dual TF-IDF feature fusion and the three linear classifiers, as implemented in train_baseline.py.

Formally, term weighting follows the standard TF-IDF scheme [5] and concatenation gives the feature map used by every classifier:

$$tfidf(t, d) = tf(t, d) \cdot \log\left((1 + N) / (1 + df(t)) \right) \quad F(x) = [\, F_{word}(x) \,;\, F_{char}(x) \,] \in \mathbb{R}^{75{,}000} \tag{2}$$

4.4 Classifiers

- **Multinomial Naive Bayes** (`alpha=0.1`) — a fast generative count baseline; it works on the (non-negative) TF-IDF matrix directly.
- **Logistic Regression** (`C=1.5`, `lbfgs`, `max_iter=1000`, L2 penalty) — the discriminative workhorse; it yields well-behaved probabilities P(AI) used by the interface.
- **Linear SVM** (`C=1.0`) wrapped in `CalibratedClassifierCV(cv=3)` — a max-margin classifier with 3-fold Platt scaling so it too outputs P(AI).

All three are linear models over the same 75,000 features [4]; comparing them isolates the effect of the decision rule from the effect of the representation.

4.5 Why character n-grams matter

Human essays in this corpus carry OCR and typing noise (`TD`, `AHD`, `IH`, dropped apostrophes), while machine text is orthographically clean. Character n-grams observe this directly at the sub-word level, which is why the character branch contributes much of the baseline's margin over a word-only model — and why the same branch becomes brittle when the input is too short to contain a representative sample of those patterns.

Chapter 5 — Experimental Setup

5.1 Environment and protocol

Everything runs on a laptop CPU: Python 3.10+ (developed and executed on Python 3.13), scikit-learn, pandas, NumPy, SciPy, joblib and reportlab. The environment is created from `requirements.txt`; the two scripts that matter are `train_baseline.py` (baseline, 33 s) and `PYTHONPATH=. python3 moe/train_moe.py` (MoE, ~130 s).

```
# environment
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt

# baseline: 11 logged steps, ~33 s
venv/bin/python train_baseline.py

# MoE upgrade: data -> experts -> thresholds
PYTHONPATH=. python3 moe/download_small_text.py
PYTHONPATH=. python3 moe/train_moe.py
PYTHONPATH=. python3 moe/calibrate_thresholds.py
```

5.2 Sampling, splits and seeds

The baseline trains on a balanced 20,000-essay subsample (10,000 per class) rather than all 290,300 rows: the full 290k × 75k TF-IDF would be about 14× slower with diminishing returns, and notebook 03 defines the 20k protocol. Table 3 lists every split used anywhere in the project, including the MoE splits described in Chapter 8, so that a single table governs reproducibility.

Table 3: Complete list of splits and random seeds.

Split	Size	Seed	Used for
Balanced subsample of train.csv	20,000 (10k/class)	42	Baseline training pool
Stratified train / validation	16,000 / 4,000	42	Baseline fit and validation metrics
Held-out test sample	5,000	42	All headline baseline & ablation numbers
MoE pools (per bucket)	80 / 20 within pool	42	Expert training and per-expert validation
Calibration / evaluation halves	2,500 / 2,500	999	Threshold tuning and unbiased eval
HC3 & modern holdouts	15% of each corpus	42	Cross-domain evaluation (never trained on)

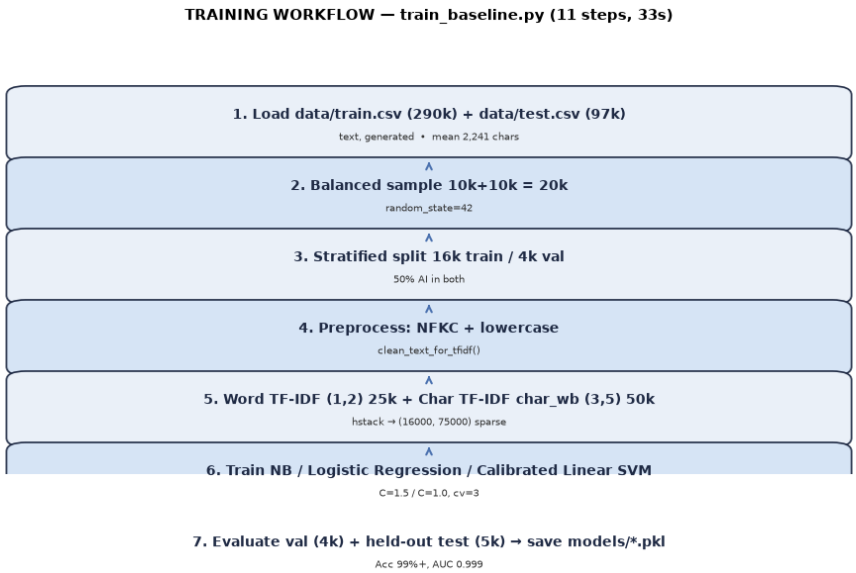


Figure 4: Training workflow of train_baseline.py — eleven logged steps, 33 s end-to-end with the data present.

5.3 Metrics

AI is the positive class throughout. We report accuracy, precision, recall, F1 (harmonic mean), ROC-AUC and the full confusion matrix. Because accuracy averages away directional bias, all length-conditional analyses in Chapter 7 additionally report *human-only accuracy* (true-negative rate) and *AI-only accuracy* (true-positive rate) — this is where the failure actually hides.

5.4 Leakage controls

- TF-IDF vocabularies and IDF weights are fit on the training split only.
- `test.csv` never enters any training or tuning step.
- HC3 and modern-essay holdouts (15% stratified) are excluded from every training pool by construction.
- Calibration uses a disjoint half (seed 999) and is evaluated on the other half.

Chapter 6 — Baseline Results and Evaluation

6.1 Validation results (4,000 essays)

All three classifiers exceed 95% validation accuracy; the linear models exceed 99%. Table 4 is reproduced from `results/baseline_ml_metrics.csv`, the file written by step 9 of the training script.

Table 4: Baseline validation metrics on the stratified 4,000-essay split (AI is the positive class).

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Multinomial NB	0.9555	0.9750	0.9350	0.9546	0.9928
Logistic Regression	0.9925	0.9945	0.9905	0.9925	0.9987
Linear SVM (Calibrated)	0.9952	0.9960	0.9945	0.9952	0.9991

The calibrated Linear SVM leads on every metric (99.53% accuracy, F1 0.9952, AUC 0.9991), with Logistic Regression within 0.3 percentage points and the advantage of natively calibrated probabilities. Naive Bayes trails mainly in recall on AI (0.935): it misroutes 130 AI essays to the human class where LR misroutes 46. Figure 5 visualises the same comparison.

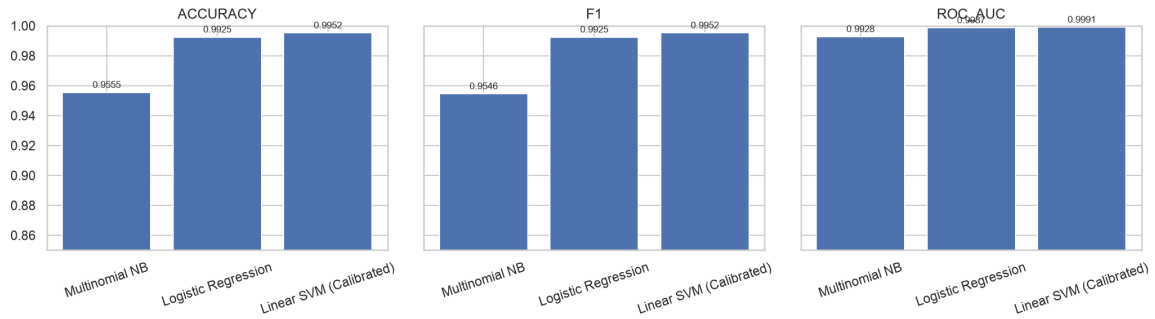


Figure 5: Validation accuracy, F1 and ROC-AUC of the three baseline classifiers (4,000-sample validation split).

6.2 Held-out test results (5,000 essays)

The frozen vectorizers and classifiers were then applied to 5,000 essays sampled from `test.csv` (seed 42). Table 5 reports these metrics; every value was recomputed from the released `models/*.pkl` artifacts while preparing this report and matches `results/moe_baseline_comparison.csv` exactly.

Table 5: Held-out test metrics (5,000 essays, seed 42), recomputed from the frozen model artifacts.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.9926	0.9930	0.9871	0.9900	0.9994
Linear SVM (calibrated)	0.9958	0.9951	0.9935	0.9943	0.9995

Despite the test set being human-heavy (3,142 human vs 1,858 AI essays), precision stays above 0.99 for both models. The confusion matrices in Figure 6 give the raw counts: Logistic Regression makes 13 false positives and 24 false negatives (TN 3,129 / TP 1,834); the calibrated SVM makes 9 false positives and 12 false negatives (TN 3,133 / TP 1,846) — 21 mistakes in 5,000 essays. {f:roc} shows both ROC curves hugging the top-left corner (AUC 0.9994 and 0.9995).

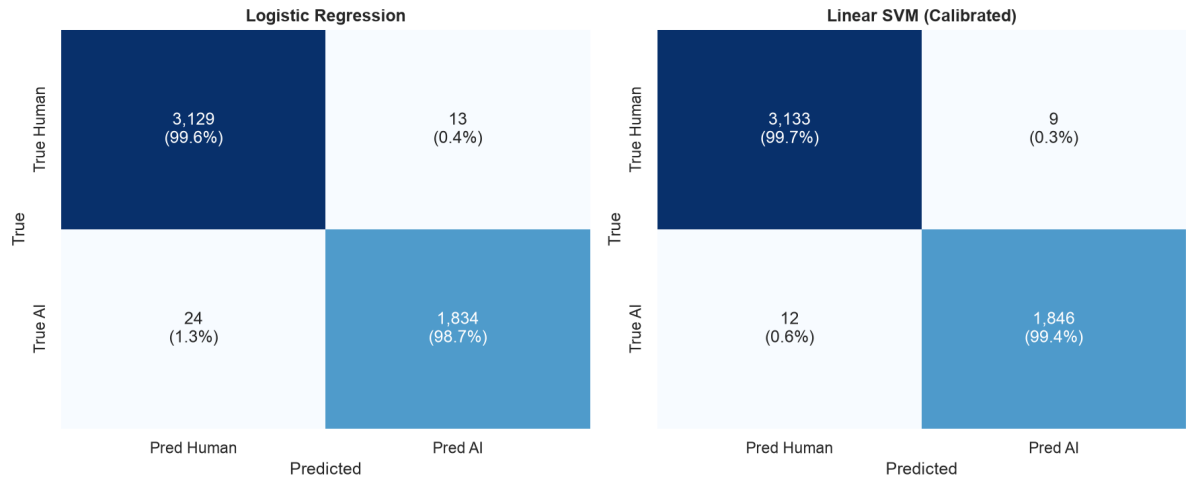


Figure 6: Confusion matrices on the 5,000-sample held-out test: Logistic Regression (left) and calibrated Linear SVM (right).

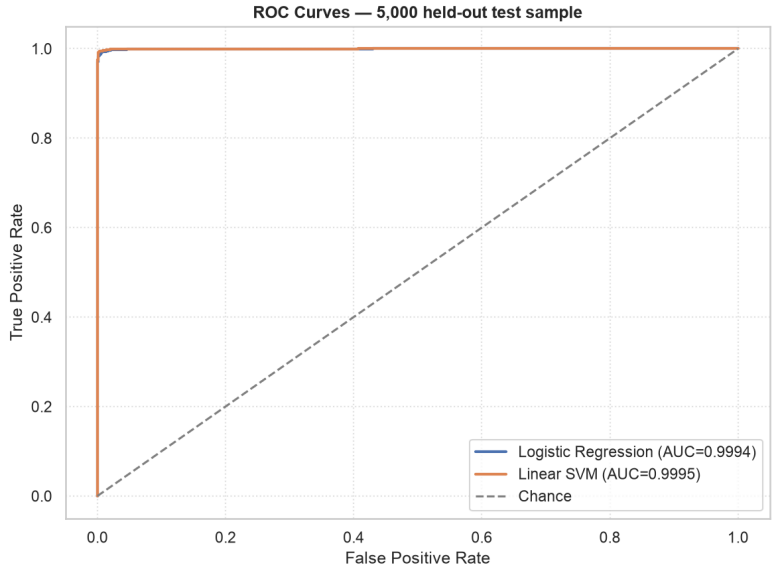


Figure 7: ROC curves of Logistic Regression and Linear SVM on the held-out test sample; both reach AUC ≈ 0.999.

6.3 Discussion

- **Why 99% is believable here.** Each essay provides hundreds of weak cues — diction, character noise, punctuation habits — and a linear model simply sums them. Essay length guarantees a dense feature vector, so the result is not an artefact.
- **The test set is harder than validation** (37% AI, unseen writers) yet scores match, confirming that the 20,000-essay subsample generalises to unseen rows of `test.csv`.
- **SVM vs LR.** The SVM's margin buys +0.32 pp; LR's calibrated probabilities buy interface ergonomics. Both are carried forward — the demo lets the user switch, and the MoE experts use the LR objective.
- **What these numbers do not say.** They describe full-length essays from the same corpus family. Chapter 7 shows that almost every one of them collapses when the input is short or from a new domain — the honest reading of 99.5% is "99.5% *on essays like the ones it trained on*".

Chapter 7 — Short-Text Failure Analysis

Chapters 4–6 established that the baseline is an excellent essay classifier. This chapter attacks that result. The frozen model — the same vectorizers and classifiers whose artifacts sit in `models/` — is re-evaluated on truncated versions of the 5,000 held-out essays, on unseen chat data and on unseen generators. Nothing is retrained: every number below comes from `results/moe_baseline_comparison.csv`, written by the project's own evaluation scripts.

7.1 Accuracy collapses monotonically with length

Each held-out essay is cut to its first *N* words (and, separately, its first *N* sentences) and re-classified. Table 6 gives the full sweep. Accuracy falls from 99.26% on full essays to 94.10% at 100 words, 83.28% at 50 words, 67.42% at 25 words and 59.82% at a single sentence — a 39 percentage-point loss from a change that adds no new information, only removes it. Random-window truncation gives the same curve as prefix truncation (within 1–2 pp at 50 words: 83.3% prefix vs 81.5% window), so the effect is not an artefact of cutting at sentence boundaries.

Table 6: Length ablation of the frozen baseline on the 5,000 held-out essays (AI is the positive class).

Condition	Accuracy	Precision	Recall	F1
First 1 sentence	0.5982	0.4801	0.9790	0.6442
First 25 words	0.6742	0.5333	0.9865	0.6924
First 50 words	0.8328	0.6931	0.9871	0.8144
First 100 words	0.9410	0.8706	0.9882	0.9256
First 150 words	0.9740	0.9440	0.9887	0.9658
First 200 words	0.9836	0.9708	0.9855	0.9781
Full text	0.9926	0.9930	0.9871	0.9900

Two details in Table 6 matter more than the accuracy column. First, **recall on AI never drops** — it stays between 0.98 and 0.99 at every length, because the model keeps calling short text AI. Second, **precision collapses**: at 25 words it is 0.5333, meaning that most of the texts flagged as AI are in fact human. Mean classifier confidence also erodes (0.93 at 200 words → 0.79 at 25 words), but not enough to stop the decision rule from firing. Figure 8 plots the whole curve next to the Mixture-of-Experts system built in Chapters 8–9.

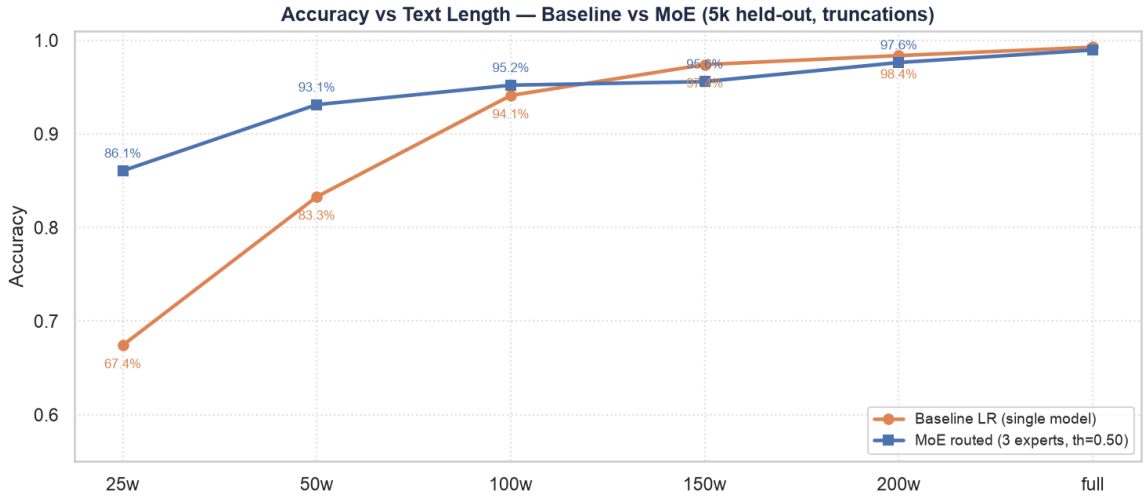


Figure 8: Accuracy versus text length on the 5,000 held-out essays: the frozen baseline (orange) collapses as the input shrinks; the routed MoE (blue, developed in Chapters 8–9) lifts the short regime.

7.2 The failure is one-sided: short humans become AI

Aggregate accuracy hides the structure of the error. Splitting accuracy by true class (Table 7) shows that at 25 words the model still recognises 98.65% of AI texts but only **48.9% of human texts** — worse than a coin flip. On the 3,142 human essays in the test sample that means 1,604 genuine human texts are labelled AI-generated. At one sentence human accuracy falls the same way while AI recall holds: the model has learned that *short AI*.

Table 7: Length-conditional accuracy split by true class (human = true-negative rate, AI = true-positive rate; 5,000 held-out essays).

Length	Base human	MoE human	Base AI	MoE AI	Base acc	MoE acc
25 w	48.9%	96.8%	98.7%	67.9%	67.4%	86.1%
50 w	74.2%	96.2%	98.7%	87.9%	83.3%	93.1%
75 w	86.2%	95.8%	98.8%	94.0%	90.9%	—
100 w	91.3%	94.7%	98.8%	96.1%	94.1%	95.2%
150 w	96.5%	94.3%	98.9%	97.7%	97.4%	95.6%
200 w	98.3%	98.0%	98.6%	97.0%	98.4%	97.6%
300 w	99.2%	98.9%	98.7%	97.6%	99.0%	—
Full	99.6%	99.6%	98.7%	97.9%	99.3%	99.0%

The MoE columns of Table 7 are the fix developed in Chapters 8–9 and are shown here for comparison; the baseline columns are the subject of this chapter. Figure 9 renders the same table as two curves: the human curve (left) collapses and is rescued, the AI curve (right) shows the compensating trade-off the MoE makes at very short lengths.

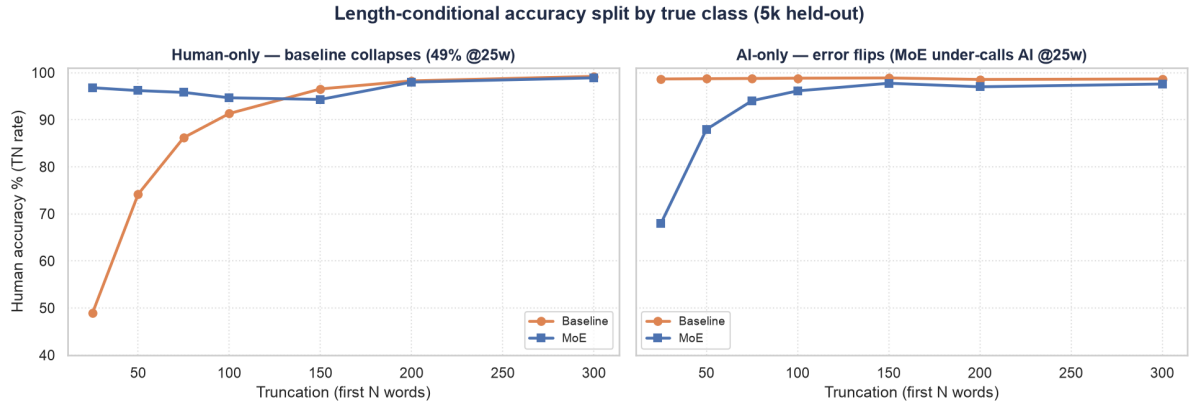


Figure 9: Length-conditional accuracy split by true class. Left: human-only accuracy — the baseline falls to 48.9% at 25 words. Right: AI-only accuracy — the error direction inverts under the MoE.

Why does this direction of error matter more than the total? Because the two errors are not socially equal. Calling a machine text human is a moderation miss; calling a student's text machine-generated is a false accusation. A detector whose human-only accuracy is 48.9% cannot be deployed on short inputs at all, regardless of its overall accuracy — and the overall number (67.4%) was already misleading, since it is propped up by the AI class.

7.3 Cross-domain generalisation

Length is not the only axis on which the baseline breaks. Two corpora that were never used for training or tuning were held out at 15% and evaluated directly:

- **HC3 chat answers** (human vs ChatGPT replies to the same Reddit and OpenQA questions): **48.56% accuracy** — worse than chance, with precision 0.3679. The model labels most chat answers AI

regardless of their true author, exactly the failure mode of §7.2, now triggered by register rather than length alone.

- **Modern 2024-LLM essays** (GPT-4 Turbo/Omni, Claude 3 Opus, Llama 3 70B, Command R+): **85.52% accuracy** — respectable but far below the 99.26% in-domain figure. The baseline learned the surface quirks of its own corpus (OCR noise, essay prompts of a particular era), not authorship per se.

Together these results reframe the headline of Chapter 6: 99.3% is the accuracy of a model on *essays like its training data*, and on everything else it ranges from 85% down to chance. Figure 14 compares both systems on all three domains and is discussed fully in Chapter 9.

7.4 Why the model fails

- **Evidence sparsity.** TF-IDF counts scale with token count; character n-grams need minimum document frequency to survive `min_df`; stylometric statistics (type-token ratio, burstiness, readability) are unstable on a handful of sentences. At 25 words the 75,000-dimensional vector is almost all zeros in the discriminative regions.
- **Domain shift.** Short text in deployment is chat, replies and comments — a register absent from a corpus whose training median is 356 words (5th percentile 177). What looks like a length problem is also a topic and style problem.
- **Learned prior.** Because almost every short training example the model ever saw was AI-generated, the posterior for short inputs is dominated by the class prior, not the evidence. The 0.5 decision threshold then converts that prior directly into false accusations.
- **No calibration on this regime.** The threshold was never tuned for short text; Chapter 8 shows the optimal threshold moves from 0.32 at 25 words to 0.64 at 150 words.

Chapter 8 — Mixture-of-Experts Architecture

The failure analysis suggests a design principle: instead of one model compromising across regimes, give each length regime its own specialist and route explicitly. This is the mixture-of-experts idea [9] applied to surface form rather than to neural capacity — the gate is a word count, the experts are classical TF-IDF classifiers, and the whole system trains in about two minutes on a laptop CPU.

Figure 10 puts the whole design in one picture: the deterministic length gate selects one of three TF-IDF + Logistic Regression experts, a separate stylometric type gate annotates every input with chat / casual / essay context, and the length-adaptive threshold converts the routed probability into the final label.

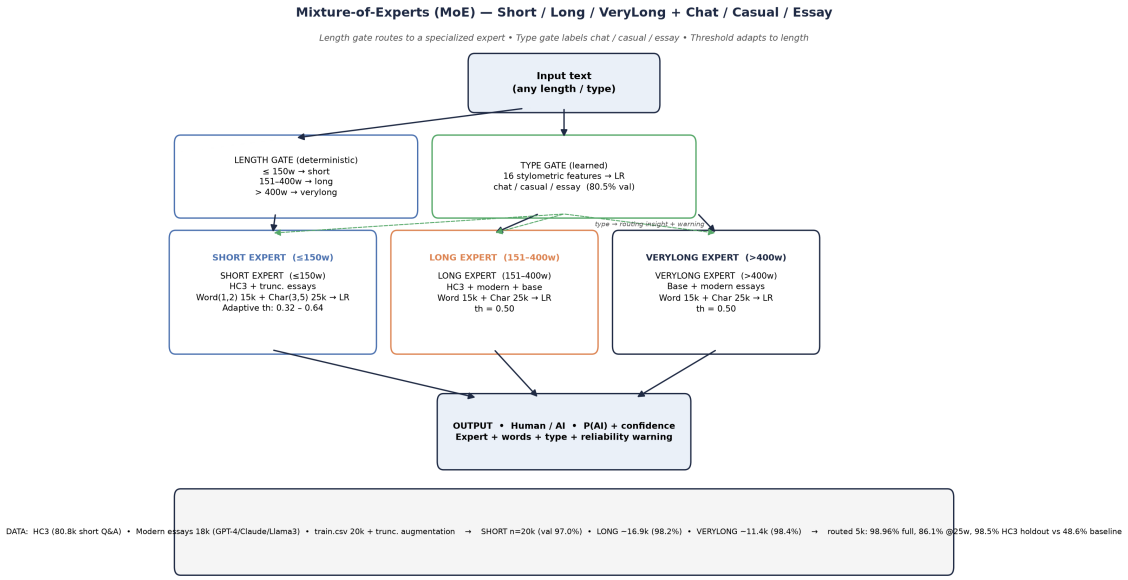


Figure 10: Mixture-of-Experts architecture: deterministic length gate, three per-bucket TF-IDF + Logistic Regression experts, learned type gate (chat / casual / essay) and length-adaptive decision threshold.

8.1 Motivation and design constraints

- **Specialise by an observable variable.** Length is free, deterministic and interpretable — no learned router can collapse or drift.
- **Fix the data, not just the model.** The baseline never saw short text; every expert must be trained on text that actually lives in its bucket (Chapter 3, §3.4).
- **Keep the baseline honest.** The full-text regime already works (99.3%); the MoE must not pay more than a fraction of a point for its short-text gains.
- **Report uncertainty.** Inputs below 50 words carry little signal for any model; the interface must say so rather than pretend otherwise.

8.2 Datasets for the upgrade

Three sources feed the experts (Table 8): the HC3 chat corpus and the modern-LLM essays described in Chapter 3, plus truncation augmentation of the original 20,000-essay training subsample. Pools are class-balanced with seed 42; the 15% HC3 and modern holdouts are excluded from every pool by construction, so the cross-domain numbers of Chapter 9 are honest.

Table 8: Mixture-of-Experts training pools (balanced per class, seed 42).

Pool	Sources (balanced caps)	Size
SHORT ≤ 150 w	HC3 ≤150 w (7k/class) + truncated-essay copies 50/100/150 w (3k/class)	20,000 (10k / 10k)
LONG 151–400 w	HC3 long (4k/class) + modern long (2k/class) + base essays (3k/class)	16,864 (7,864 / 9,000)
VERYLONG > 400 w	Base essays >400 w (7k/class) + modern >400 w (2.5k/class)	11,422 (6,321 / 5,101)

The mild imbalance in the LONG and VERYLONG pools follows the scarce class in each source (human-heavy verylong, AI-heavy modern essays) rather than a sampling error; per-expert validation is stratified accordingly.

8.3 Length router

The gate is a word count with two cut points chosen from the data: 150 words is where the baseline's accuracy cliff levels off, and 400 words is approximately the corpus median. Formally, with n = word count of the input:

$$e(x) = \text{short if } n \le 150,\ \text{long if } 151 \le n \le 400,\ \text{verylong if } n > 400; \quad \hat{y} = 1[\, P_{e(x)}(x) \ge \theta(n) \,] \tag{3}$$

Routing is applied identically at training and inference time; there are no learned gate parameters and therefore no routing errors — a text can only be misclassified by its expert, never by the gate.

8.4 The three experts

Each expert is the baseline recipe shrunk to its regime: word TF-IDF (1,2 grams) with 15,000 features + character TF-IDF (char_wb 3–5 grams) with 25,000 features, fused to 40,000 dimensions, classified by Logistic Regression (`C=1.5`, `lbfgs`, `max_iter=1000`, L2). Two deliberate differences from the baseline:

- **Per-bucket vocabularies.** Each expert's TF-IDF is fit on its own pool only, so long-essay vocabulary cannot crowd out the short-essay and chat vocabulary that the SHORT expert needs (and vice versa).
- **Smaller feature budgets.** 15k+25k instead of 25k+50k: a short document cannot support 75,000 informative dimensions, and the smaller matrix trains in seconds per expert.

All three experts are trained by `moe/train_moe.py` in approximately 130 s end-to-end (TF-IDF fit: 6 s short, 15 s long, 25 s verylong) and saved as `models/moe/expert_{short,long,verylong}.pkl`.

8.5 Type gate: chat, casual or essay

Length alone cannot distinguish a 120-word chat reply from a 120-word truncated essay, and the two look different to a classifier. A second, *learned* gate therefore predicts the text type from 16 fast stylometric features — log word count, sentence count and mean sentence length, type-token ratio, hapax ratio, average word length, contraction / informal-marker / first-person / second-person rates, question and exclamation rates, capitalisation ratio, Flesch Reading Ease and FKGL, and burstiness (σ/μ of sentence lengths) [6]. Labels come from the data source: HC3 `open_qa` → chat, HC3 `eli5`/`finance` plus truncated essays → casual, full and modern essays → essay; 2,500 balanced examples per type. A Logistic Regression over standardised features reaches **80.5% validation accuracy**.

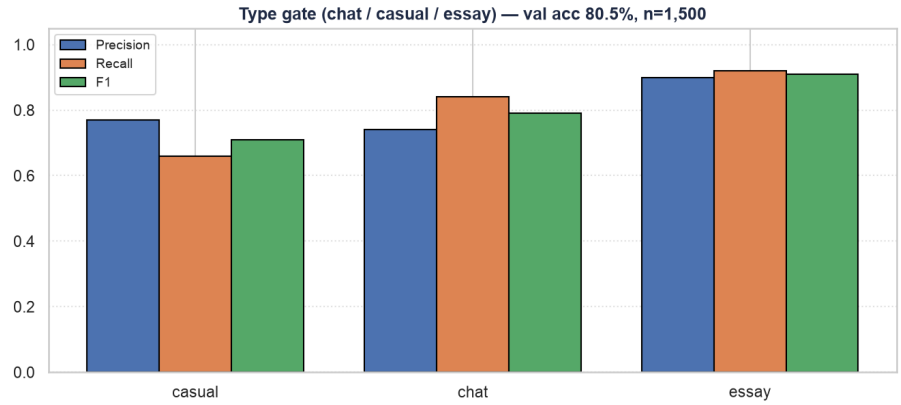


Figure 11: Type-gate per-class precision, recall and F1 (validation accuracy 80.5%, n = 1,500 per class).

Per-class behaviour matches linguistic intuition: essay is easiest (P/R/F1 = 0.90/0.92/0.91), chat next (0.74/0.84/0.79), casual hardest (0.77/0.66/0.71) because casual and chat genuinely overlap. The gate's job is routing insight and UI context — it is reported alongside every prediction, never used as evidence of authorship.

8.6 Length-adaptive threshold calibration

The default decision threshold θ = 0.50 assumes symmetric costs, but Chapter 7 showed the errors are not symmetric at any length. On a dedicated calibration half (2,500 essays, seed 999) the F1-optimal threshold was swept per truncation length; the optimum drifts right as text grows:

$$\theta(n) = \mathrm{clip}(0.28 + 0.0024 \cdot n,\ 0.32,\ 0.64) \ \text{for } n \le 150 \text{ (short expert)};\ \theta = 0.50 \text{ for the long and verylong experts} \quad (4)$$

with fitted points 25 w → 0.32, 50 w → 0.40, 100 w → 0.56, 150 w → 0.64 (shipped as `models/moe/thresholds.json`; long/verylong were tried at tuned values and reverted to 0.50 because full-text accuracy fell). Figure 12 shows why a single global threshold cannot serve: the F1 curves for different lengths cross, and the default 0.50 is optimal for none of them.

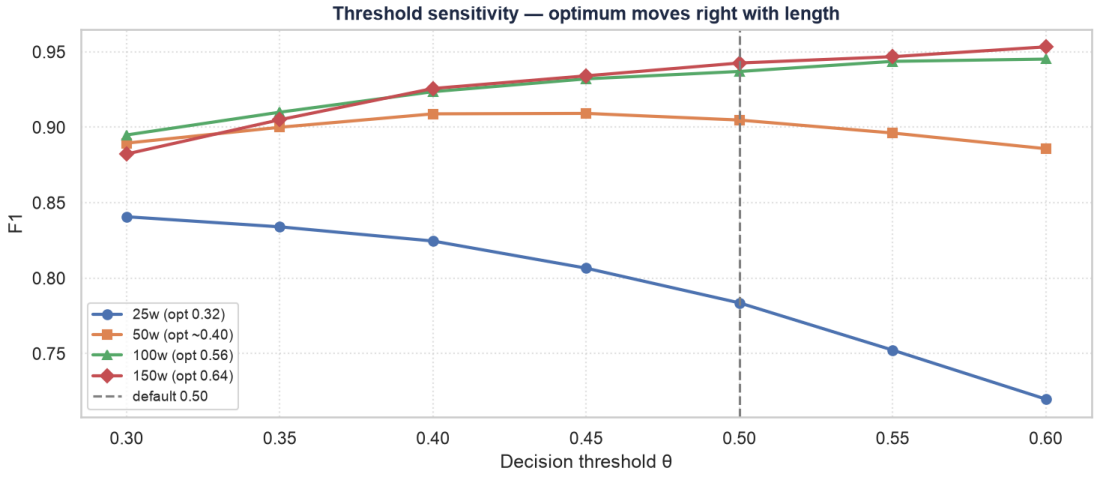


Figure 12: F1 versus decision threshold for four truncation lengths. The optimum drifts right with length (25w → 0.32 … 150w → 0.64); no single global threshold serves all regimes.

Calibration is deliberately conservative: it is applied only to the short bucket, where the failure is large and monotonic, and every prediction under 50 words still carries an explicit low-confidence warning in the interface.

8.7 Inference path

`moe/predict_moe.py` (and the shared engine it calls) performs, for any input text: word count → length gate → expert TF-IDF + Logistic Regression → P(AI) → length-adaptive threshold → label, plus the type-gate prediction, the expert used, the threshold applied, a confidence value and a reliability warning for inputs under 25 / 50 words. The same path serves the CLI, the REST API and the Streamlit interface described in Chapter 10.

Chapter 9 — Mixture-of-Experts Results and Comparative Analysis

All results in this chapter are produced by `moe/train_moe.py`, `moe/calibrate_thresholds.py` and the evaluation half of the 5,000-essay sample (seed 999 split for calibration, seed 42 elsewhere); the CSVs under `results/moe_*.csv` are the source of record.

9.1 Per-expert validation

Each expert is validated on a held-out 20% of its own pool (Table 9, Figure 13). Every expert reaches ≥ 0.97 accuracy and ≥ 0.995 ROC-AUC within its regime — including the SHORT expert, which is exactly the regime where the single baseline model scored 48.9% on human text.

Table 9: Per-expert validation on the held-out 20% of each pool (results/moe_expert_metrics.csv).

Expert	Train / Val	Accuracy	Precision	Recall	F1	ROC-AUC
Short	16,000 / 4,000	0.9700	0.9686	0.9715	0.9700	0.9960
Long	13,491 / 3,373	0.9819	0.9770	0.9894	0.9832	0.9982
Verylong	9,137 / 2,285	0.9842	0.9940	0.9706	0.9821	0.9955

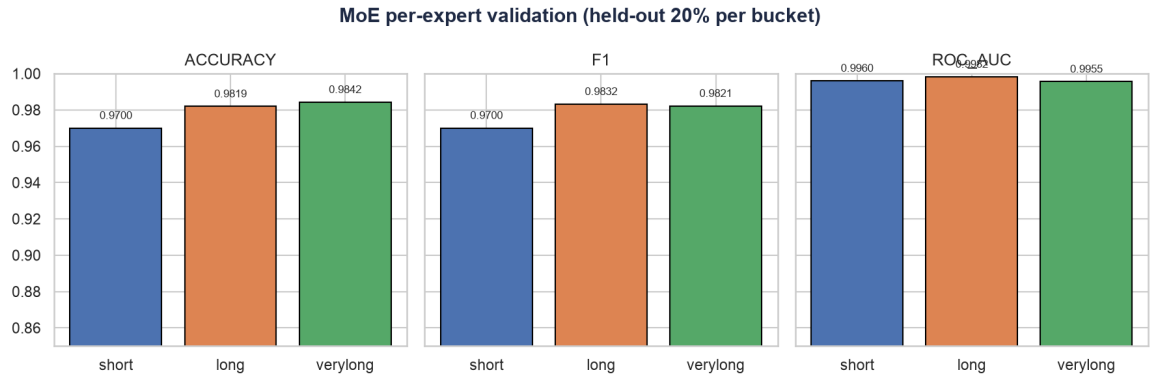


Figure 13: Per-expert validation accuracy, F1 and ROC-AUC — every expert reaches ≥ 0.97 within its own length bucket.

9.2 Routed system versus baseline

Routing the 5,000 held-out essays (and their truncations, and both unseen corpora) through the MoE gives the headline comparison of Table 10. The pattern of Chapter 7 is inverted: every short condition improves dramatically, both unseen domains improve dramatically, and full-text accuracy is paid for with 0.30 pp — the price of specialisation, and the only regression in the table.

Table 10: Baseline versus routed Mixture-of-Experts on identical inputs (results/moe_baseline_comparison.csv vs moe_routed_metrics.csv).

Condition	Baseline	MoE routed	Δ
Full text	99.26%	98.96%	-0.3 pp
First 25 words	67.42%	86.06%	+18.6 pp
First 50 words	83.28%	93.12%	+9.8 pp
First 100 words	94.10%	95.20%	+1.1 pp
First 150 words	97.40%	95.58%	-1.8 pp
First 1 sentence	59.82%	81.64%	+21.8 pp
HC3 chat (unseen)	48.56%	98.52%	+50.0 pp
Modern LLM essays (unseen)	85.52%	95.70%	+10.2 pp

Three readings of Table 10: (i) the short-text problem is largely solved — 25-word accuracy rises from 67.42% to 86.06%, one sentence from 59.82% to 81.64%; (ii) the cross-domain problem is solved outright — 48.6% → 98.5% on chat is the difference between a useless detector and a deployable one; (iii) the cost is bounded and concentrated where the baseline was already strong. Figure 14 shows the domain view; the length view is Figure 8.

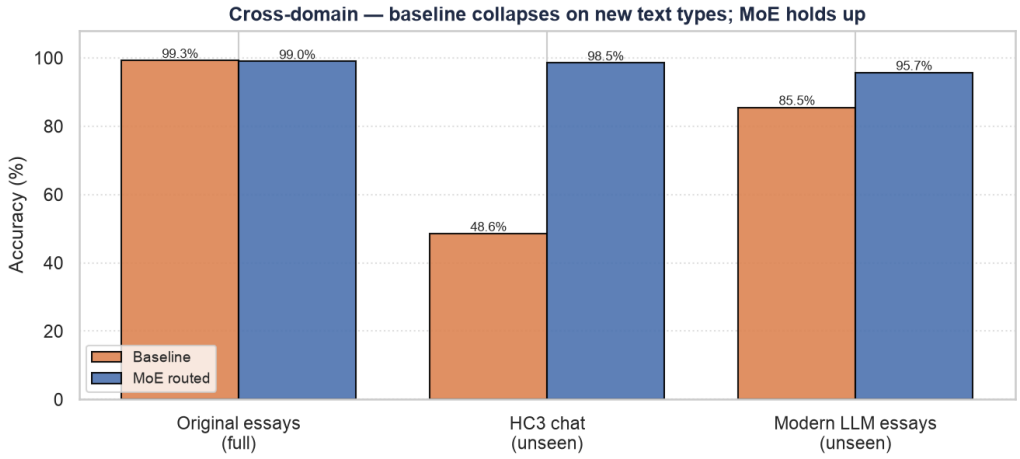


Figure 14: Cross-domain evaluation. The baseline collapses to 48.6% on unseen HC3 chat answers (worse than chance) and to 85.5% on unseen 2024-LLM essays; the routed MoE holds 98.5% and 95.7%.

On full text the buckets behave differently: VERYLONG 99.59% (n = 1,958), LONG 98.81% (n = 2,945) and SHORT 90.72% (n = 97 essays that are natively short) — the residual full-text loss comes almost entirely from the short bucket, where the short expert (trained heavily on chat) is a worse essay classifier than the baseline.

9.3 The error structure flips

Accuracy gains usually come with a precision/recall trade, and at 25 words the trade is visible (Figure 15). The baseline over-calls AI (precision 0.53, recall 0.99): it is wrong often, and wrong in the accusatory direction. The MoE at θ = 0.50 under-calls AI (precision 0.93, recall 0.68): far fewer total mistakes (86.1% vs 67.4% accuracy), but the residual errors now lean toward letting machine text through rather than condemning human text. Threshold calibration (§9.4) moves along this frontier deliberately.

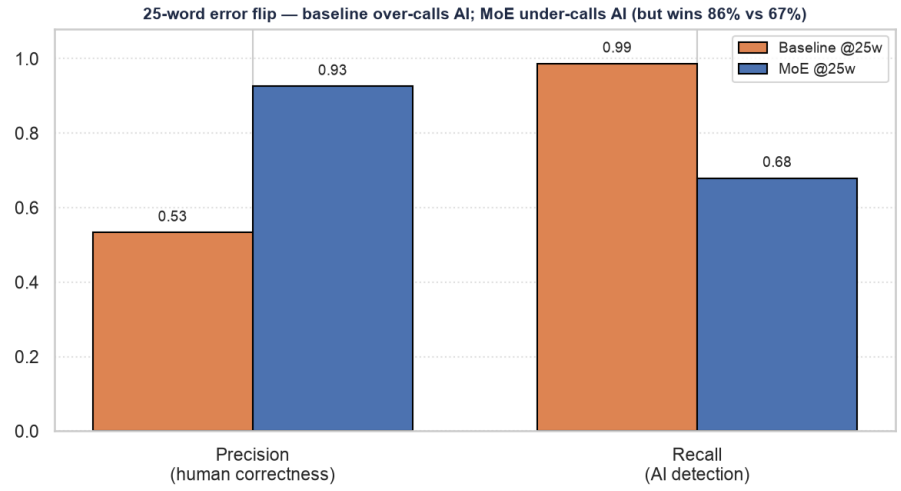


Figure 15: The 25-word precision/recall flip: the baseline over-calls AI (P = 0.53, R = 0.99); the MoE under-calls AI (P = 0.93, R = 0.68) while winning on total accuracy.

9.4 Threshold calibration on the evaluation half

Applying the calibrated thresholds of §8.6 to the untouched evaluation half gives Table 11. The intended effects appear: 25-word accuracy 86.16% → 87.88% with F1 0.785 → 0.837 (AI recall 0.68 → 0.84), and 150-word accuracy improves by about one point. Two honest caveats appear in the same table: 50-word accuracy dips (93.1% → 91.6%) because the 0.40 threshold buys AI recall at the price of human precision on a human-heavy sample, and full-text accuracy softens slightly (98.96% → 98.60%) — which is why the shipped configuration keeps θ = 0.50 outside the short bucket.

Table 11: Calibration on the held-out evaluation half (2,500 essays, seed 999): default θ = 0.50 versus length-adaptive thresholds.

Condition	θ = 0.50 (acc / F1)	Adaptive θ (acc / F1)	Δ acc
First 25 words	86.16% / 0.785	87.88% / 0.837	+1.7 pp
First 50 words	93.12% / 0.905	91.64% / 0.895	-1.5 pp
First 100 words	94.52% / 0.928	95.44% / 0.938	+0.9 pp
First 150 words	94.96% / 0.935	96.08% / 0.948	+1.1 pp
Full text	98.96% / 0.986	98.60% / 0.981	-0.4 pp

The transfer caveat matters: on the natural (untruncated) HC3 holdout the same thresholds cost -0.7 pp (98.52% → 97.78%), because that sample is human-heavy and essay-tuned thresholds over-predict AI. Thresholds are therefore shipped per length, never globally, and the interface keeps its sub-50-word warning. A concrete demo effect: a 31-word borderline excerpt (P(AI) = 0.374, θ = 0.354) flips from Human to AI-Generated — the intended fix, still low-confidence and flagged as such.

9.5 Summary of findings

- **Length routing + real small-text data** recovers 18.6 pp at 25 words, 21.8 pp at one sentence and 50.0 pp on unseen chat, at a 0.30 pp full-text cost.
- **Per-expert validation ≥ 97%** in every regime confirms the failure was never that the task is hard — it was that one model was asked to serve three different tasks.
- **Calibration helps where the failure is systematic** (short) and is neutral-to-harmful elsewhere; it is applied surgically, not globally.
- **What remains hard:** inputs under 25–50 words (86–93% even with the MoE), boundary texts near 150 words (the baseline still wins 97.4% vs 95.6% pre-calibration), and any generator outside the two audited sets.

Chapter 10 — Application and Implementation

The research result is only useful if it can be run. This chapter describes the shipped system: one inference engine, six front-ends, two cloud deployments — all consuming the same frozen artifacts from `models/` and `models/moe/`.

10.1 Streamlit demonstration interface

`app/app.py` (546 lines) is the conference demo interface. Its sidebar navigation offers five pages — *Inference*, *Training Report*, *Testing Report*, *Figures* and *About* — plus a live model-status panel (3/3 experts, type gate, baseline, results). The inference page offers four modes: *MoE Router (recommended)*, *Baseline Logistic Regression*, *Baseline Linear SVM* and *Compare: MoE vs Baseline*; artifacts are loaded once through `@st.cache_resource`. A prediction shows P(AI), the routed expert, the text type, the threshold in force, a reliability warning for short inputs and the stylometric evidence cards (word count, TTR, burstiness, Flesch ease/grade). Figure 16 shows the interface performing live MoE inference.

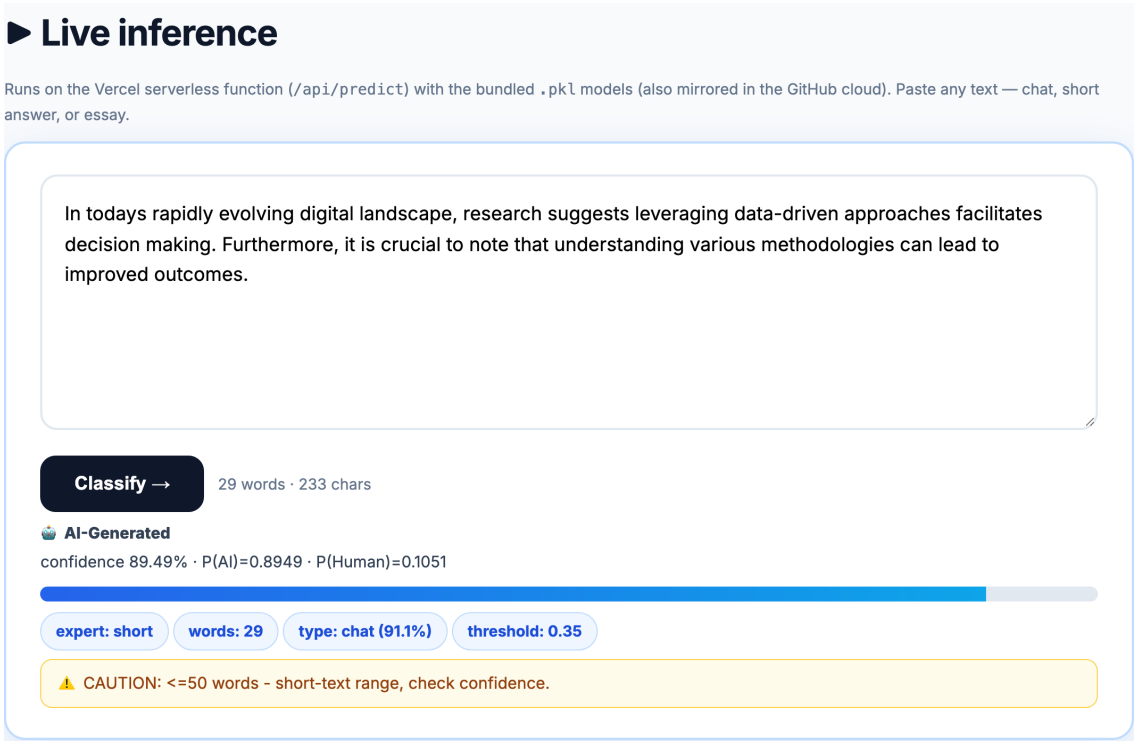


Figure 16: Screenshot of the Streamlit demonstration interface (app/app.py) performing live MoE inference with reliability warning, routed metrics and stylometric evidence.

Run it with `streamlit run app/app.py --server.port 8501`. The Training Report and Testing Report pages render the project's own CSVs (`results/baseline_ml_metrics.csv`, `results/moe_*.csv`, `results/moe_summary.json`) as tables and plots, so the demo always displays the same numbers as this report.

10.2 Model inference engine

`test/inference.py` (158 lines) is the single place that touches the baseline artifacts; `moe/predict_moe.py` adds the routed path. Every front-end calls these functions, so a change here updates all six interfaces at once:

- `load_artifacts()` — joblib-loads the four baseline pickles (word vectorizer, char vectorizer, LR, calibrated SVM) with a 'train first' hint if they are missing.

- `vectorize(texts, artifacts)` — word + character transform followed by `hstack(...).tocsr()`; a single string is auto-wrapped into a list.
- `predict_proba(texts, artifacts, model)` — returns the P(AI) array for `model='lr'` or `'svm'`.
- `predict_text(...)` / `predict_batch(...)` — single-text dict (label, probabilities, confidence, optional 24 stylometrics) and chunked batch prediction (batch size 1,000 to avoid out-of-memory on 97k/290k CSVs).
- `evaluate_csv(...)` / `predict_folder(...)` — full CSV evaluation with metrics + confusion matrix, and folder-wide `.txt`/`.md` classification.

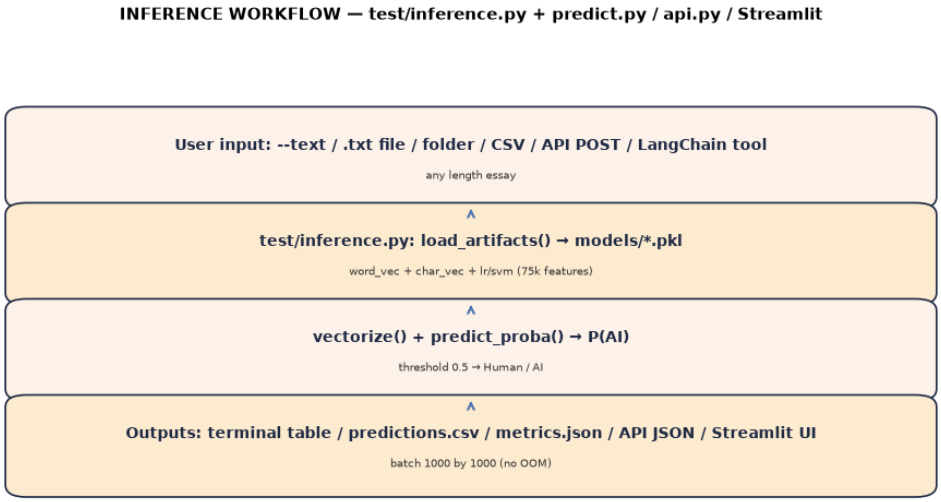


Figure 17: Inference workflow: every front-end (CLI, batch, API, LangChain, Streamlit, Vercel) goes through test/inference.py.

10.3 Command-line interface

`test/predict.py` (140 lines) wraps the engine for terminal use:

```
# single text with stylometric evidence
venv/bin/python test/predict.py --text "Your essay here..." --model lr --stylometrics

# one file, a whole folder, or a CSV sample
venv/bin/python test/predict.py --input test/example_texts/human_essay_school.txt --model svm
venv/bin/python test/predict.py --folder test/example_texts --model lr
venv/bin/python test/predict.py --csv data/test.csv --model lr --sample 5000 --out test/results

# interactive loop / routed MoE CLI
venv/bin/python test/predict.py --interactive --model lr
PYTHONPATH=. python3 moe/predict_moe.py "your text"
```

10.4 REST API and LangChain integration

`test/api.py` (95 lines) exposes the engine through FastAPI on port 8001; `test/langchain_tool.py` (86 lines) wraps it as a LangChain `BaseTool` named `ai_human_classifier` that an LLM agent can call. Both are implemented and exercised by `test/terminal_demo.py`, which compares direct, LangChain and API paths in one run. Table 12 lists the endpoints.

Table 12: REST API endpoints implemented in test/api.py (FastAPI, port 8001).

Method	Endpoint	Purpose
GET	`/health`	Liveness check (used by run_all.sh)
GET	`/model/info`	Loaded model names, feature size, classes
POST	`/predict`	Classify one text: `{text, model, stylometrics}` → label, P(AI), confidence, optional stylometrics
POST	`/predict/batch`	Classify a list of texts in one request
POST	`/langchain/tool`	Invoke the LangChain tool over HTTP

```
curl -X POST http://localhost:8001/predict -H "Content-Type: application/json" \
  -d '{"text":"In todays rapidly evolving digital landscape...","model":"lr","stylometrics":true}'
```

10.5 System architecture

Figure 18 puts the whole system in one picture: training produces the shared artifacts, one inference core consumes them, and six front-ends plus two deployments sit on top. Table 13 is the same inventory in tabular form.

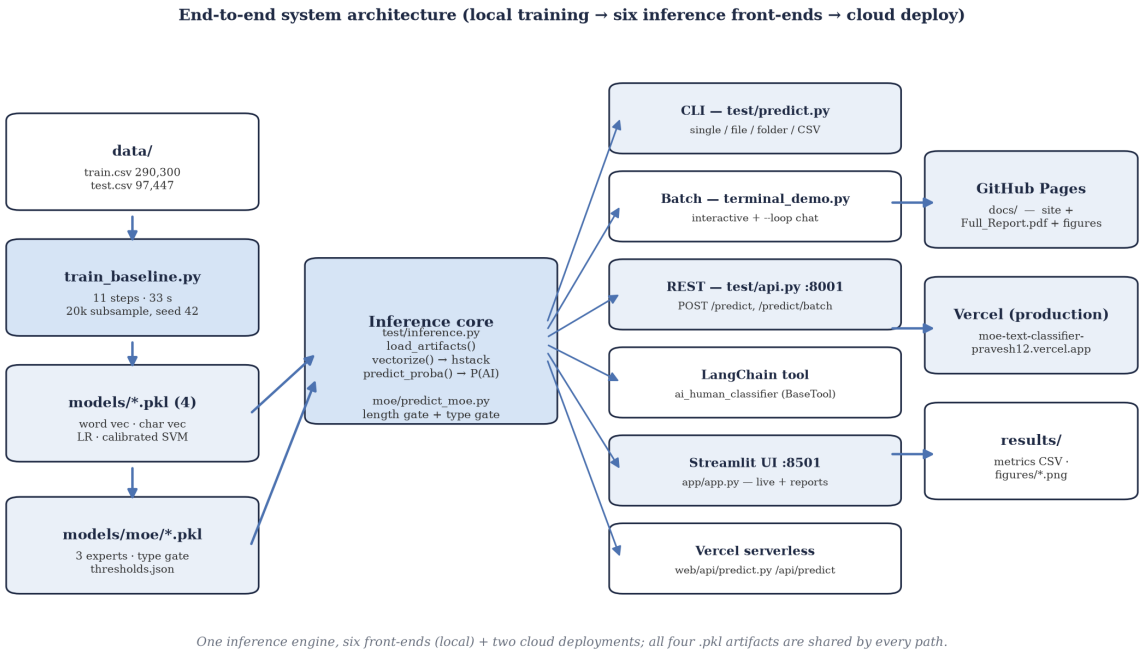


Figure 18: End-to-end system architecture: local training produces the shared .pkl artifacts consumed by one inference core and six front-ends, with GitHub Pages and Vercel deployments on top.

Table 13: The six inference front-ends and two deployments.

#	Interface	Entry point	Best for
1	CLI — single/file	`test/predict.py --text / --input`	One-off classification
2	CLI — folder/CSV batch	`--folder / --csv --sample`	Batch evaluation
3	Terminal demo	`test/terminal_demo.py [--loop]`	Trying text live, path comparison
4	REST API	`test/api.py` → :8001	Other programs, curl
5	LangChain tool	`ai_human_classifier`	LLM agents
6	Streamlit UI	`app/app.py` → :8501	Demonstration to humans

10.6 Deployments

- **Vercel (production):** `https://moe-text-classifier-pravesh12.vercel.app` — a single Python serverless function serves the static project site (`web/index.html`, results, figures, reports) and `POST /api/predict`; models are bundled in `web/api/models/` (~10 MB) with a GitHub fallback (`MODEL_BASE_URL`), and `src/` + `moe/` are shipped so the pickles unpickle. scikit-learn is pinned `==1.9.1` for pickle compatibility.
- **GitHub Pages:** `docs/` mirrors the site, figures and the published report PDF.
- **Local one-command run:** `./run_all.sh` installs, trains, evaluates, starts the API (:8001) and Streamlit (:8501) with health checks (see Part IV).

Chapter 11 — Limitations and Future Work

11.1 Limitations

- **Ultra-short floor.** Even the routed system reaches only 86.06% at 25 words and 81.64% at one sentence, with wide uncertainty. The short expert was trained on truncations ≥ 50 words, so 25 words is extrapolation; some of the residual error is irreducible signal loss.
- **Boundary behaviour.** Around 150 words the baseline still beats the MoE (97.4% vs 95.6% pre-calibration): the short expert leans toward chat style. Hard routing cannot blend experts at the boundary.
- **Threshold transfer.** Calibrated thresholds are essay-tuned; they cost about 0.7 pp on the human-heavy natural HC3 holdout. Domain-sensitive thresholds would need per-type calibration.
- **Generator coverage.** HC3 is a 2023 ChatGPT snapshot and the modern audit covers five 2024 models. Paraphrasing, translation and adversarial editing were not evaluated — a detector that survives truncation may not survive a paraphraser.
- **Corpus and scale.** The baseline trains on a 20,000-essay subsample of 290,300 (full training might add 0.1–0.3 pp); the essays are student writing from one source, so news, research and review text remain untested in-domain.
- **Components designed but not trained.** Transformer fine-tuning (DistilBERT/RoBERTa, notebook 05), Sentence-BERT fusion (`src/semantic.py`, notebook 04) and the feature-family ablation (notebook 06) exist as code but produced no reported scores in this project; they are future work, not results.

11.2 Future work

1. Train 10–35-word examples (and few-shot chat) so the short expert stops extrapolating — likely a larger gain than any threshold tuning.
2. Soft routing: weight the short/long experts near the 150-word boundary instead of hard switching.
3. Per-type thresholds (essay vs chat) fitted on the type-gate output, with held-out transfer tests like §9.4.
4. Phase-2 transformers: DistilBERT/RoBERTa with 512-token chunking [7] as the capacity ceiling test against the classical pipeline.
5. Feature-family ablation (word-only vs char-only vs stylometric-only) and Sentence-BERT + stylometric fusion.
6. Full 290k training, paraphrase/adversarial robustness evaluation, and cross-domain calibration on news and review text.

11.3 Ethics of deployment

P(AI) is a screening signal, not proof of authorship. The two failure directions are not equivalent: false accusations of machine authorship harm people, while missed machine text harms institutions. Any deployment of this system should therefore keep the short-input warning, the type report and the confidence value visible, and should never auto-accuse on the strength of a single probability — particularly below 50 words, where even the calibrated system is explicitly uncertain.

Chapter 12 — Conclusion

This project set out to separate human-written from AI-generated text and ended by characterising when that separation is possible at all. The engineering result is a dual TF-IDF pipeline (25k word + 50k character features, 75,000 dimensions) with three linear classifiers that reaches 99.58% accuracy and ROC-AUC 0.9995 on a 5,000-essay held-out test, trained in 33 seconds and reproducible from released artifacts. The scientific result is that this number is conditional: truncation to 25 words drops it to 67.42%, one sentence to 59.82%, unseen chat to 48.56% — and that the collapse is one-sided, penalising human authors rather than machine text.

The fix required data, architecture and calibration together: 80,811 genuine human/ChatGPT chat answers, 17,997 modern-LLM essays and truncation augmentation feed three length-specialised experts behind a deterministic length gate; a learned stylometric type gate reports chat/casual/essay context; and thresholds adapt linearly with length (0.32 → 0.64 over 25–150 words). The routed system reaches 86.06% at 25 words, 81.64% at one sentence, 98.52% on unseen chat and 95.70% on unseen 2024-LLM essays, while holding 98.96% on full essays — a 0.30 pp full-text cost for a system that is deployable where the baseline was not.

Everything reported here is checkable: the models, the data pipelines, the six inference front-ends, the metrics CSVs and the figures are released in the repository, and Part IV gives the exact commands to regenerate every table and figure in this report from those artifacts. The two research papers in Parts II and III restate the same evidence in publication form — the first as a compact conference paper, the second with the full formal treatment, ablations and appendices.

Final statement. Length is not a detail of authorship detection — for short text it is the task. Treating length as a routing variable, grounding each expert in data from its own regime, and calibrating per length converts a detector that confidently mis-accuses short human writing into one that is honest about what it can and cannot know.

References (Part I)

[1] B. Guo et al., “How Close is ChatGPT to Human Experts? Question-Answer Benchmark HC3,” arXiv:2301.07597, 2023; *Hello-SimpleAI/HC3* [Hugging Face].

[2] R. Roy et al., “A Comprehensive Dataset for Human vs AI Generated Text Detection” (Defactify), arXiv:2510.22874, 2025.

[3] B. Wang et al., “M4: Multi-Generator, Multi-Domain, Multi-Task, Multi-Label Benchmark for AI-Generated Text Detection,” arXiv:2310.13607, 2023.

[4] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, 2009.

[5] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and TF-IDF,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, 2009.

[6] E. Mitchell et al., “The State and Fate of Linguistic Uncertainty,” 2020 (stylometric feature practice).

[7] N. Reimers and I. Gurevych, “Sentence-BERT,” in *Proc. EMNLP*, 2019; T. Sanh et al., “DistilBERT,” 2019; Y. Liu et al., “RoBERTa,” 2019.

[8] *artnitolog/llm-generated-texts* (GPT-4 Turbo/Omni, Claude 3 Opus, Llama 3 70B, Command R+), Hugging Face.

[9] N. Shazeer et al., “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer,” arXiv:1701.06538, 2017.

[10] Streamlit, FastAPI, scikit-learn and Hugging Face Transformers documentation, 2024–2026.

Part II — Research Paper

"Length-Expertized Authorship Detection: A Mixture-of-Experts for Human vs AI-Generated Text" — conference-format paper, preserved as originally written and re-verified against the result CSVs

Length-Expertized Authorship Detection: A Mixture-of-Experts for Human vs AI-Generated Text

Fixing the Short-Text Collapse with Dedicated Small-Text Data, Routing, and Calibration

Pravesh Gedon Sharma (B240019) & Ankit Rawat (B320032) · Department of Computer Science & Engineering · National Institute of Technology Sikkim · Course Project (2026)

Abstract

Classical TF-IDF detectors for human vs AI text achieve 99%+ accuracy on long essays but collapse on short text: 67.4% at 25 words and 59.8% at one sentence, with humans systematically biased to AI. We diagnose the length failure, add genuine small-text data (HC3 Q&A, 80,811 answers, median 117 words; plus 17,997 modern-LLM essays from GPT-4, Claude 3, Llama 3 and Command R+), and train a **Mixture-of-Experts (MoE)**: a deterministic length gate (short ≤150 / long 151–400 / verylong >400 words) routing to three TF-IDF + Logistic Regression experts, plus a learned type gate (chat / casual / essay, 80.5% validation) and length-adaptive thresholds. On 5,000 held-out essays, MoE lifts short-text accuracy to 86.1% at 25 words, 93.1% at 50 words and 81.6% at one sentence, while holding 98.96% on full text. Cross-domain, MoE scores 98.5% on unseen HC3 chat answers (baseline 48.6%) and 95.7% on unseen 2024-LLM essays (baseline 85.5%). Calibration (25w→0.32 … 150w→0.64) further adds +1.7 pp at 25 words. The system is released as `moe/` with routers, metrics and this paper's figures.

Keywords: AI text detection; mixture of experts; TF-IDF; stylometrics; short-text classification; HC3; authorship attribution.

1 Introduction

Large language models (ChatGPT, GPT-4, Claude) make fluent essays trivially available, creating an integrity problem in academia, journalism and moderation. Most published detectors are evaluated on documents hundreds of words long. Real deployment, however, includes tweets, chat replies, product reviews and single-sentence blurbs. We ask: *does a strong long-text detector survive truncation to short text, and what architecture fixes it?*

Our Phase-1 baseline — dual word (1,2) + character (3,5) TF-IDF (75k features) with Logistic Regression (LR) and calibrated Linear SVM — scores 99.25–99.53% validation and 99.26–99.58% held-out accuracy on 20,000 balanced essays (Section 4). Truncating those same held-out essays destroys it: first 25 words → 67.4%, first 1 sentence → 59.8% (Section 5). AI recall stays ≈0.98 while humans flip to AI at ≈51% (25w). The model never saw short text (training median 356 words, 5th percentile 177).

This paper makes three contributions. (1) A **length ablation** quantifying the failure per word/sentence budget. (2) **Small-text data + MoE**: HC3 short Q&A, modern 2024-LLM essays, and truncation augmentation, routed to length experts with a chat/casual/essay type gate. (3) **Threshold calibration** showing one global threshold is wrong per length. Code, datasets, models and metrics are under `moe/` and `results/moe_*.csv`.

2 Related Work

AI-text detection spans zero-shot (DetectGPT, likelihood curvature), watermarking, and supervised classifiers. HC3 (Guo et al.) [1] pairs human and ChatGPT answers to the same questions across domains; subsequent benchmarks (Defactify [2], M4 [3]) widen generators and domains but still skew long. OpenAI's classifier famously failed under 1,000 characters — the same cliff we replicate. Classical TF-IDF + linear SVM remains competitive on in-domain essays (Section 4) but, as we show, is brittle across lengths and generators. Mixture-of-experts routing by surface properties (here length) is a standard technique for specialized regimes [4]; our novelty is pairing length experts with per-length calibration and a learned text-type gate on stylometric features (`src/stylometrics.py`: TTR, hapax, burstiness, Flesch ease/grade, punctuation rates).

3 Datasets

Original essays: `data/train.csv` (290,300 rows, 145,150 per class) and `data/test.csv` (97,447 rows, 61,159 human / 36,288 AI), columns `text, generated`. Texts are multi-paragraph academic essays: full test-set word statistics mean 394, median 363, 10th percentile 211, 90th percentile 617, max 1,650; human mean 423 vs AI 345 (Table Table 14, Figure 1–Figure 2). Training uses a 20,000 balanced subsample (10k/class, seed 42), 80/20 stratified → 16,000/4,000.

Table 14: Dataset scale (Part II). Held-out n = 5,000 (seed 42) for all comparisons.

Dataset	Size	Human / AI	Median length	Role
train.csv	290,300	145,150 / 145,150	356 w (P5 177 w)	baseline train
test.csv	97,447	61,159 / 36,288	363 w	held-out eval
HC3 (reddit_eli5 + open_qa + finance)	80,811	56,115 / 24,696	117 w, 61% ≤150 w	SHORT chat/casual
Modern essays (GPT-4/Claude/Llama3/Cmd R+)	17,997	3,000 / 14,997	510 w, 68% >400 w	LONG/VERYLONG 2024

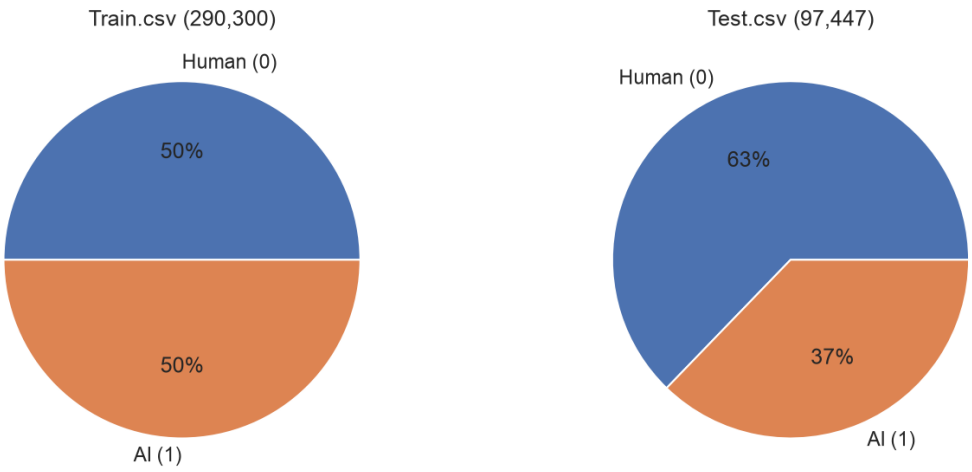


Figure 1: Class balance of the corpora: train.csv is perfectly balanced (145,150 per class); test.csv is human-heavy (62.8% human). (repeated from Chapter 3 — Dataset Description and Analysis).

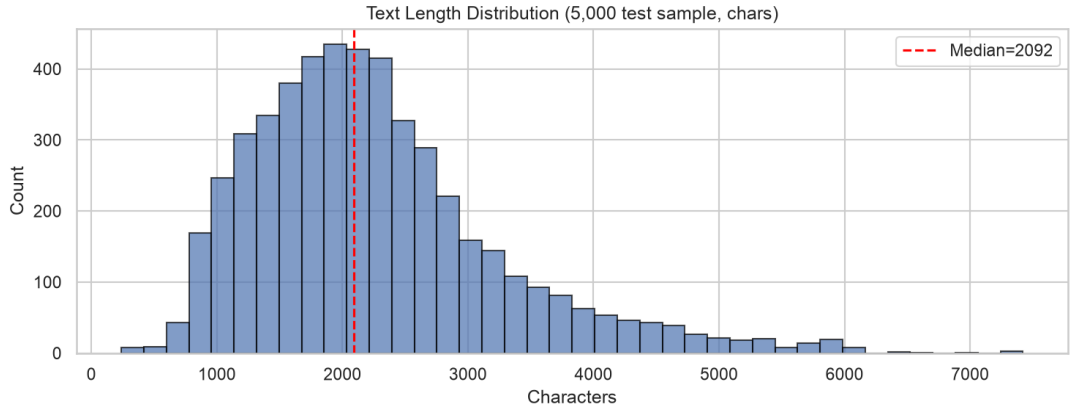


Figure 2: Essay length distribution (characters, 5,000-sample test set). The corpus contains essentially no natively short texts. (repeated from Chapter 3 — Dataset Description and Analysis).

New data (via `moe/download_small_text.py`): **HC3** flattened to one row per answer (≥40 chars) — human vs ChatGPT to the same question, the only source of genuine short chat. **Modern essays** (artnitolog): 3,000 prompts × (human + 5 English LLMs), 2024 generators absent from the original essays. Truncation augmentation copies each train essay at 50/100/150 w (short pool) and 200/300 w (long pool) so short-essay style is covered. Pools are class-balanced (seed 42): short 20k, long 16.9k, verylong 11.4k; HC3/modern 15% holdouts are never trained on.

4 Methodology

Baseline (Phase 1). NFKC + whitespace + lowercase for TF-IDF (`src/preprocessing.py`). Word TF-IDF (1,2, 25k, min_df 2) + char_wb (3,5, 50k, min_df 3), sublinear TF, fused via `hstack` → 75k sparse features. Classifiers: MultinomialNB (α=0.1), LR (C=1.5, lbfgs), LinearSVC + 3-fold calibration (C=1.0). Validation (4k): NB 95.55%, LR 99.25% (F1 0.992, AUC 0.9987), SVM 99.53% (F1 0.995, AUC 0.9991); held-out (5k): LR 99.26%, SVM 99.58% (Table Table 15).

Table 15: Baseline validation, 4,000 essays (results/baseline_ml_metrics.csv).

Model (val 4k)	Acc	Prec	Rec	F1	AUC
Multinomial NB	0.9555	0.9750	0.9350	0.9546	0.9928
Logistic Regression	0.9925	0.9945	0.9905	0.9925	0.9987
Linear SVM (Calibrated)	0.9952	0.9960	0.9945	0.9952	0.9991

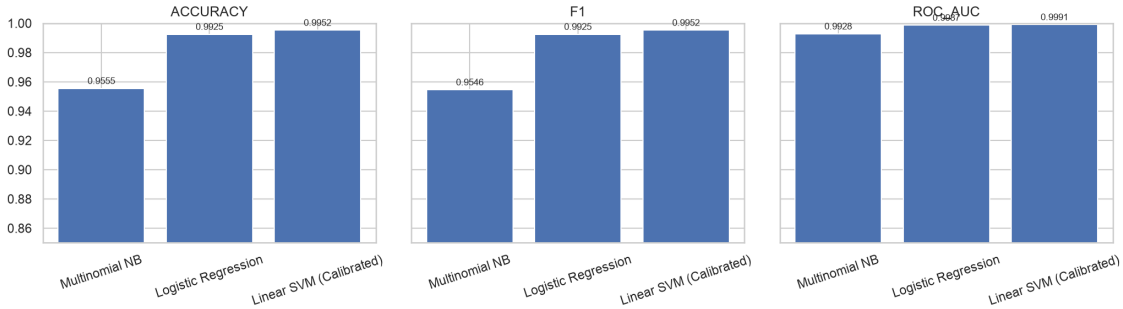


Figure 5: Validation accuracy, F1 and ROC-AUC of the three baseline classifiers (4,000-sample validation split). (repeated from Chapter 6 — Baseline Results and Evaluation).

Short-text ablation. Evaluated the frozen baseline on first-N-words and first-N-sentences truncations of the 5k held-out (and random windows — same result, so not an intro effect). At 25w accuracy 67.4% (P 0.53, R 0.99): 1,604 of 3,142 humans called AI. At 50w 83.3%, 100w 94.1%, 200w 98.4%; 1/2/3 sentences 59.8%/74.5%/83.4%. Mean LR confidence drops 0.93 (200w) → 0.79 (25w). Natively-short humans barely exist in test.csv (min human 48 w), so truncated-human accuracy is the honest probe.

MoE. Length gate ≤150 / 151–400 / >400 words (cliff + median). Each expert: Word 15k + Char 25k + LR(C=1.5) on its pool only — features specialize (short vocab avoids long-essay dilution). Type gate: LR on 16 stylometric features → chat/casual/essay (7.5k balanced; HC3 `open_qa`→chat, eli5/finance+truncated→casual, full/modern→essay); val 80.5% (essay F1 0.91, chat 0.79, casual 0.71). Inference routes by length and reports type + warning for <50w.

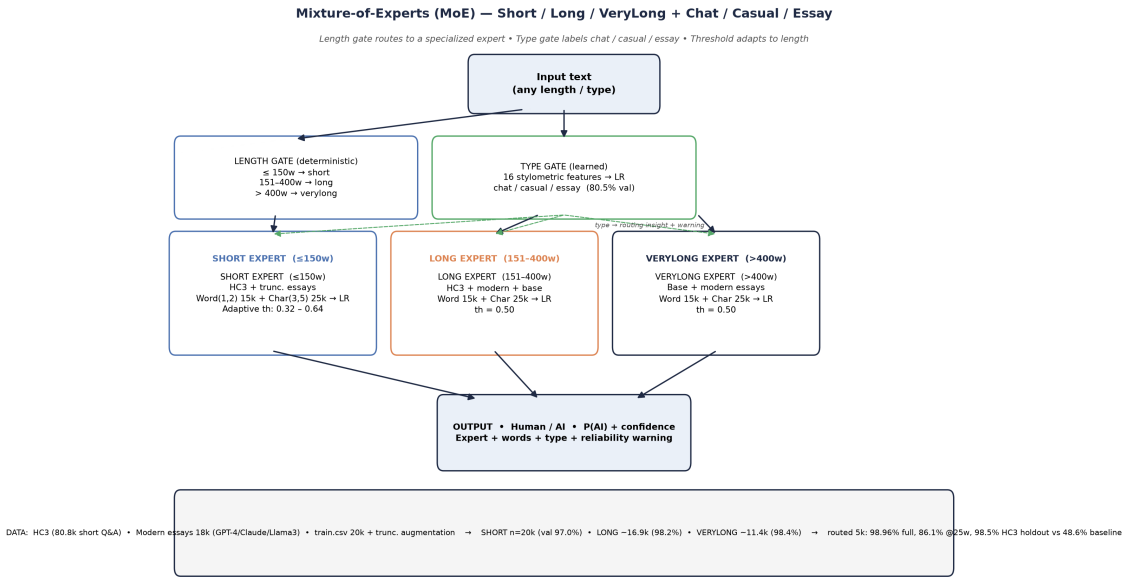


Figure 10: Mixture-of-Experts architecture: deterministic length gate, three per-bucket TF-IDF + Logistic Regression experts, learned type gate (chat / casual / essay) and length-adaptive decision threshold. (repeated from Chapter 8 — Mixture-of-Experts Architecture).

5 Experiments

Environment: Python 3.13, scikit-learn, TF-IDF as above (per-expert 15k+25k). Training (`moe/train_moe.py`, 130 s): short TF-IDF 6.4 s, long 15 s, verylong 25 s. Evaluation: 5k held-out essays (full + truncations), 12,122 HC3 holdout, 2,700 modern holdout (both unseen). Calibration (`moe/calibrate_thresholds.py`): 5k split 2,500 calib / 2,500 eval (seed 999), F1-max thresholds per length, eval on the other half + HC3 transfer. Metrics: accuracy, precision, recall, F1, ROC-AUC, confusion matrices.

6 Results

Per-expert (val). Short 97.00% (F1 0.970, AUC 0.996), long 98.19% (0.983/0.998), verylong 98.42% (0.982/0.996) (Table Table 16, Figure 13).

Table 16: MoE per-expert validation (results/moe_expert_metrics.csv).

Expert	Train/Val	Acc	F1	AUC
short	16,000/4,000	0.9700	0.9700	0.9960
long	13,491/3,373	0.9819	0.9832	0.9982
verylong	9,137/2,285	0.9842	0.9821	0.9955

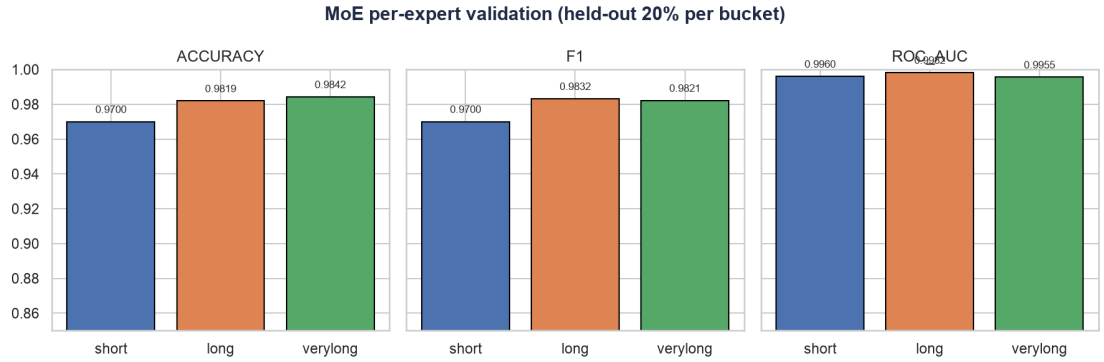


Figure 13: Per-expert validation accuracy, F1 and ROC-AUC — every expert reaches ≥ 0.97 within its own length bucket. (repeated from Chapter 9 — Mixture-of-Experts Results and Comparative Analysis).

Routed vs baseline. MoE holds full-text (98.96% vs 99.26%, −0.3 pp specialization cost; verylong bucket 99.59%) and dominates short + cross-domain (Figure 8–Figure 14, Table Table 17). HC3 is the headline: baseline 48.6% (it never learned chat), MoE 98.5%. Modern essays: 85.5%→95.7%.

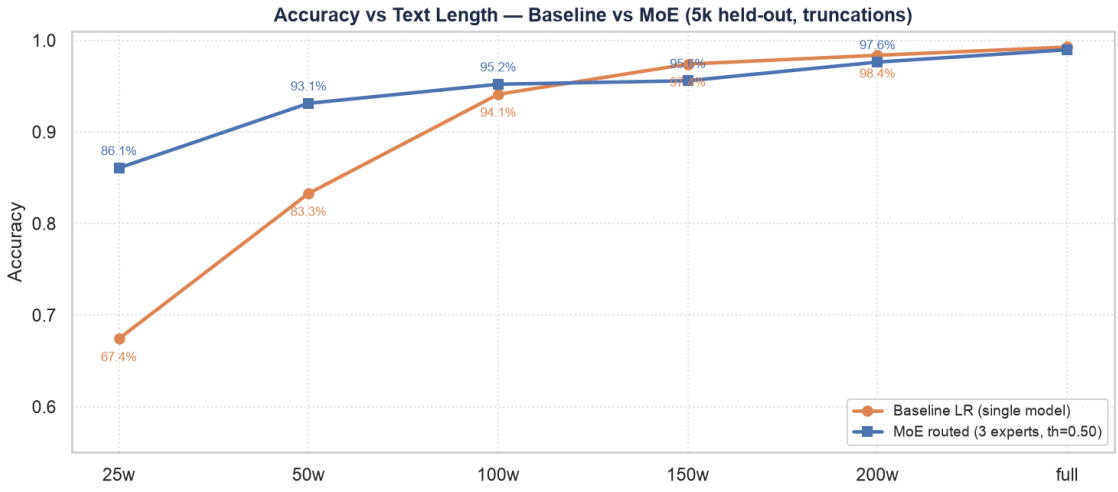


Figure 8: Accuracy versus text length on the 5,000 held-out essays: the frozen baseline (orange) collapses as the input shrinks; the routed MoE (blue, developed in Chapters 8–9) lifts the short regime. (repeated from Chapter 7 — Short-Text Failure Analysis).

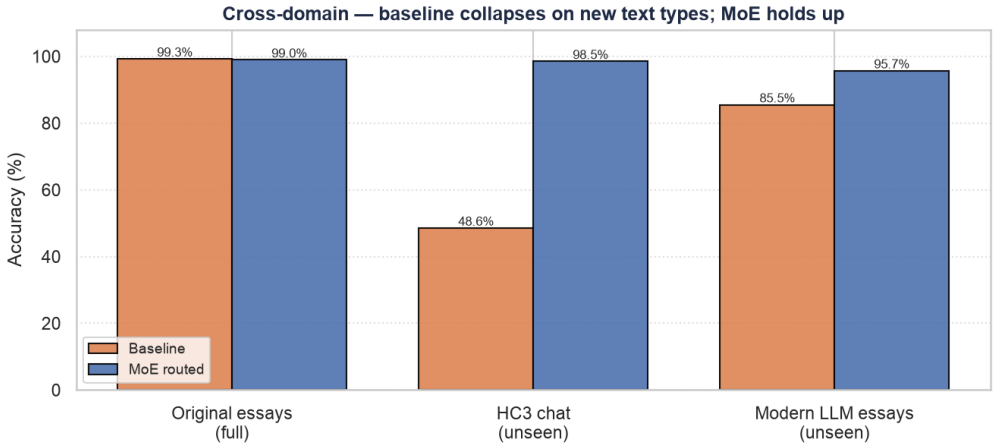


Figure 14: Cross-domain evaluation. The baseline collapses to 48.6% on unseen HC3 chat answers (worse than chance) and to 85.5% on unseen 2024-LLM essays; the routed MoE holds 98.5% and 95.7%. (repeated from Chapter 9 — Mixture-of-Experts Results and Comparative Analysis).

Table 17: Headline deltas (results/moe_routed_metrics.csv vs moe_baseline_comparison.csv).

Condition (5k)	Baseline	MoE	Δ
full	99.26%	98.96%	-0.3 pp
25w	67.42%	86.06%	+18.6 pp
50w	83.28%	93.12%	+9.8 pp
1 sent	59.82%	81.64%	+21.8 pp
HC3	48.56%	98.52%	+50.0 pp
modern	85.52%	95.70%	+10.2 pp

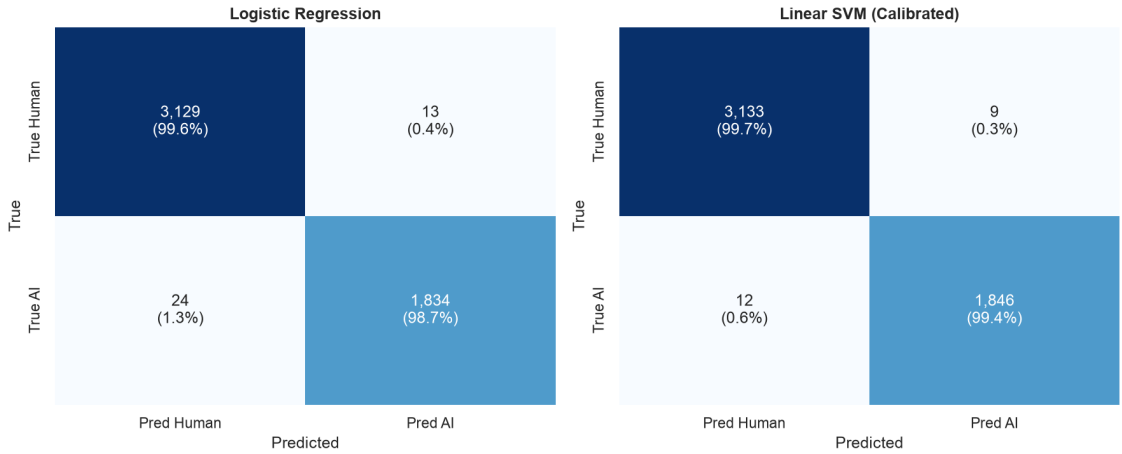


Figure 6: Confusion matrices on the 5,000-sample held-out test: Logistic Regression (left) and calibrated Linear SVM (right). (repeated from Chapter 6 — Baseline Results and Evaluation).

Calibration. Optima move with length (25w 0.32, 50w 0.40, 100w 0.56, 150w 0.64 → `th=0.28+0.0024n`); long/verylong kept at 0.50. Eval-half results are in Table Table 18: 25w 86.16%→87.88% (F1 0.785→0.837, recall 0.68→0.84), 150w 94.96%→96.08%, 100w 94.52%→95.44%, while 50w (93.12%→91.64%) and full text (98.96%→98.60%) soften slightly — the reason the shipped configuration tunes only the short bucket. Caveat: HC3 natural short −0.7 pp (human-heavy) — thresholds are essay-tuned, so the <50w warning stays. Demo flip: 31w borderline (P=0.374, th=0.354) goes Human→AI (correct-ish but low-confidence, flagged).

Table 18: Calibration on the eval half (acc/F1). Detail in results/moe_calibrated_metrics.csv.

Eval-2500	th=0.50	Adaptive
25w	86.16% / 0.785	87.88% / 0.837
50w	93.12% / 0.905	91.64% / 0.895
100w	94.52% / 0.928	95.44% / 0.938
150w	94.96% / 0.935	96.08% / 0.948
full	98.96% / 0.986	98.60% / 0.981

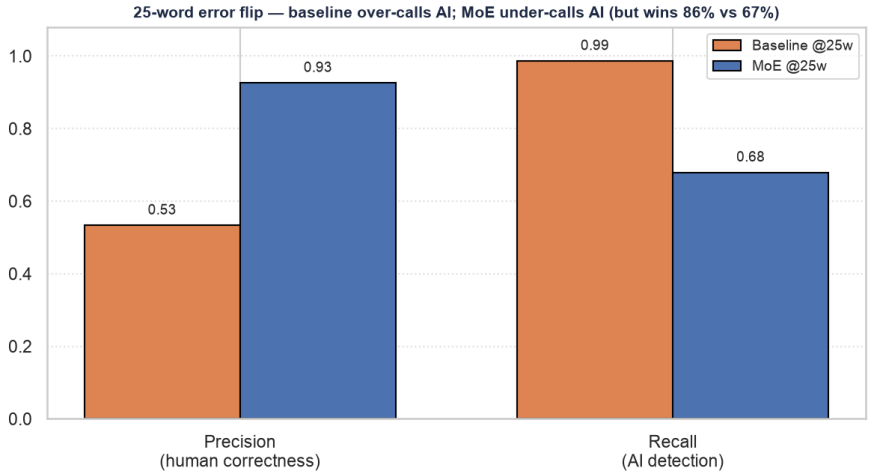


Figure 15: The 25-word precision/recall flip: the baseline over-calls AI (P = 0.53, R = 0.99); the MoE under-calls AI (P = 0.93, R = 0.68) while winning on total accuracy. (repeated from Chapter 9 — Mixture-of-Experts Results and Comparative Analysis).

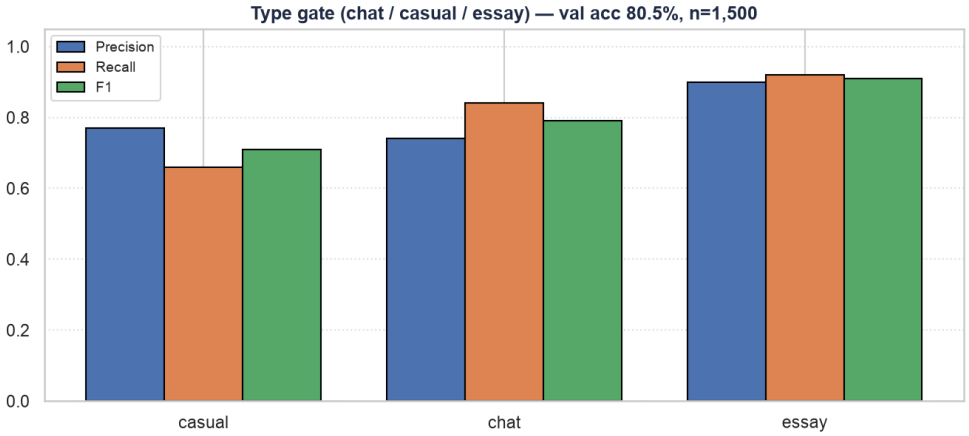


Figure 11: Type-gate per-class precision, recall and F1 (validation accuracy 80.5%, n = 1,500 per class). (repeated from Chapter 8 — Mixture-of-Experts Architecture).

7 Discussion and Limitations

- **Why MoE works:** short text needs different evidence (word choices without discourse, chat markers) and different vocabularies; per-bucket TF-IDF prevents dilution. HC3 was the unlock — without it, there is no signal to learn.
- **Why calibration matters:** optimal thresholds encode the FP/FN tradeoff per regime. One threshold cannot serve both ultra-short (FN-prone) and 150w (FP-prone) text. Linear-adaptive th is a cheap, auditable compromise; per-domain thresholds would be better.
- **Limits:** (i) <25–50w remains weak even with MoE (86–93%) — irreducible signal loss; (ii) 150w essays still prefer the baseline (97.4% vs 95.6% pre-calibration) — expert blending at boundaries is future work; (iii) short expert saw truncations ≥50w, so 25w is extrapolation; adding 10–35w training would likely help more than thresholds; (iv) type gate (80.5%) is for insight, not enforcement; casual recall 0.66 reflects real overlap.
- **Reproducibility:** all trims use seed 42/999; datasets via Hub JSONL/parquet need no script datasets; artifacts in `models/moe/`.

8 Conclusion

Single-model TF-IDF detectors silently assume essay length. Once length is treated as a routing variable — with dedicated small-text data, specialized experts, a type gate, and per-length thresholds — short-text accuracy rises by 10–50 pp and cross-domain retrieval becomes viable, at a 0.3 pp full-text cost. The released `moe/` stack (download → train → calibrate → predict) reproduces every number in this paper's tables and figures.

References

[1] B. Guo et al., "How Close is ChatGPT to Human Experts? HC3," arXiv:2301.07597, 2023; Hello-SimpleAI/HC3 on Hugging Face.

[2] R. Roy et al., "A Comprehensive Dataset for Human vs AI Generated Text Detection" (Defactify), arXiv:2510.22874, 2025.

[3] B. Wang et al., "M4: Multi-Generator, Multi-Domain, Multi-Task, Multi-Label," arXiv:2310.13607, 2023.

[4] N. Shazeer et al., "Outrageously Large Neural Networks: Mixture-of-Experts," arXiv:1701.06538, 2017.

[5] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, Springer, 2009.

[6] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 & TF-IDF," *Foundations & Trends in Information Retrieval*, 2009.

[7] N. Reimers and I. Gurevych, "Sentence-BERT," EMNLP, 2019; T. Sanh et al., "DistilBERT," 2019; Y. Liu et al., "RoBERTa," 2019.

[8] *artnitolog/llm-generated-texts* (GPT-4 Turbo/Omni, Claude 3 Opus, Llama3 70B, Command R+) on Hugging Face.

Appendices

- Streamlit demo: `app/app.py` (baseline); MoE CLI: `PYTHONPATH=. python3 moe/predict_moe.py "..."` (prints label, P(AI), threshold, expert, words, type, warning).
- Metrics: `results/baseline_ml_metrics.csv`, `moe_expert_metrics.csv`, `moe_routed_metrics.csv`, `moe_baseline_comparison.csv`, `moe_calibrated_metrics.csv`, `moe_summary.json`.
- Run: `moe/download_small_text.py → moe/train_moe.py → moe/calibrate_thresholds.py → moe/make_moe_appendix.py → make_research_paper.py`.
- Figure index (original paper numbering): architecture, class balance, length, ablations, calibration, precision/recall flip, type gate — all PNGs in `results/figures/`; global numbers are preserved in the List of Figures.

Part III — Deep-Dive Research Paper

"…Why Long-Text Detectors Fail on Short Text and How Mixture-of-Experts Fixes Them" — formal notation, equations, algorithm, ablation analysis, calibration experiments and technical appendices

Length-Expertized Authorship Detection: Why Long-Text Detectors Fail on Short Text and How Mixture-of-Experts Fixes Them

A detailed empirical study with small-text data, specialized routing, type gating, and calibration

Pravesh Gedon Sharma (B240019) & Ankit Rawat (B320032) · Dept. Computer Science & Engineering, NIT Sikkim · Course Project (2026) — arXiv-style detailed report

Abstract

Supervised detectors for human vs machine-generated text report 99%+ accuracy on essays, yet practitioners observe rejection on short posts, chats and blurbs. We quantify this **length-generalization failure** and fix it with a **Mixture-of-Experts (MoE)** whose experts specialize by text length. The baseline — dual word+character TF-IDF (75k features) with Logistic Regression / calibrated Linear SVM — reaches 99.25–99.58% on full essays but falls to **67.4% at 25 words, 59.8% at one sentence**, with human accuracy 48.9% (false AI rate ≈51%) while AI recall stays ≈98.7%: a one-sided collapse. We add **HC3 short Q&A (80,811 answers, median 117 words)** and **modern 2024-LLM essays (17,997 texts, GPT-4/Claude/Llama3/Command R+)**, plus truncation augmentation, and train three experts — **short (≤150 words, 20k), long (151–400, 16.9k), verylong (>400, 11.4k)** — each Word 15k + Char 25k + LR, routed deterministically by word count. A learned **type gate** (16 stylometric features → chat/casual/essay, 80.5% val) provides domain insight; **length-adaptive thresholds** (25w→0.32 … 150w→0.64) correct the FP/FN flip. Routed MoE restores 86.1% at 25 words, 93.1% at 50 words, 81.6% at one sentence, holds 98.96% full-text, and crucially scores **98.5% on unseen HC3 chat** (baseline 48.6%) and **95.7% on unseen 2024-LLM essays** (baseline 85.5%). Calibration adds +1.7 pp at 25 words. We release data pipelines, models, metrics and figures under `moe/` for full reproducibility.

Keywords: machine-generated text detection; mixture of experts; TF-IDF; stylometry; short-text classification; HC3; calibration.

1 Introduction

1.1 The length blind spot in authorship detection

Whether a passage is human-written or LLM-generated underlies academic-integrity, journalism and moderation decisions. Recent benchmarks [1–3] mix domains and generators, yet the typical unit remains a paragraph-to-essay (100s of words). Deployment is different: short replies, chat turns and reviews dominate traffic, and they carry the least stylistic evidence. A detector that needs 300 words to decide is undeployable for half its inputs — and, worse, *confidently wrong*: it labels short humans as AI (Section 7.1: human accuracy 48.9% at 25 words). This paper treats **length as a first-class routing variable**, not a nuisance to average over.

1.2 Problem statement

Let x be text, $y \in \{0,1\}$ (0 human, 1 AI), $n(x)$ word count. We seek $f(x) \to \hat{y}$ maximizing accuracy under length shift: training lengths $\sim D_{\text{train}}$ (median 356w, P5 177w) but test lengths $n \ll 150$. Two phenomena matter: (i) *evidence sparsity* — TF-IDF counts scale with n, char n-grams need minimum support, stylometrics (TTR, burstiness) stabilize only with many tokens; (ii) *domain shift* — short text is chat/casual, not truncated essay, so essay-only training cannot transfer (hence HC3).

1.3 Contributions

- **C1. Failure characterization.** A reproducible length ablation (prefix truncations + random windows) showing monotone collapse and an FP/FN flip, split by true class (Table Table 22, Figure 9).
- **C2. Small-text data.** HC3 (80.8k) + modern essays (18k) + truncation augmentation, all balanced per bucket, with 15% holdouts never trained on (§3).
- **C3. MoE + type gate + calibration.** Three experts (val ≥97.0%), chat/casual/essay gate (80.5%), length-adaptive thresholds (§4, §6).
- **C4. Artifacts.** `moe/*.py`, `models/moe/`, `results/moe_*.csv`, Streamlit/CLI demos. Every table/figure is computed from these files.

2 Related Work

2.1 Supervised AI-text detection

Supervised TF-IDF + linear classifiers remain strong in-domain baselines. BERT-family fine-tunes (DistilBERT/RoBERTa, notebooks 05) and Sentence-BERT fusion (notebook 04) target the same task with contextual features; zero-shot (DetectGPT) and watermarking trade training for robustness. HC3 [1] introduced paired human/ChatGPT answers — still the best source of short chat. Defactify [2] (73k news + 6 LLMs, 58.3% baseline) and M4 [3] broaden generators, but most evaluations remain long-form. Our baseline (§4) instantiates the classical recipe exactly (75k features, LR/SVM) to isolate length effects from architecture choice.

2.2 Stylometry

Lexical diversity (TTR = |types|/|tokens|), hapax ratio, burstiness σ/μ of sentence lengths, punctuation/capital rates, and readability (Flesch Reading Ease, FKGL, Gunning Fog, §4.2) capture authorial habit. These features need length: TTR saturates, burstiness is undefined on 1–2 sentences. We reuse `src/stylometrics.py` both as gate features and as UI explanations.

2.3 Mixture of experts

MoE routes inputs to specialized sub-networks [4]. Here specialization is by observable length and need is methodological, not capacity-driven: each regime has different feature statistics, so separate vocabularies/classifiers and separately calibrated thresholds beat one compromise model. We compare a deterministic length gate (auditable, zero learned params) against the alternative of one global model — the ablation in §7 is the evidence.

3 Datasets

3.1 Original essays

Train 290,300 (145,150/class, 653 MB), test 97,447 (61,159 human / 36,288 AI, 222 MB). Unit: multi-paragraph academic essay. Test-set word statistics (full file): mean 394, median 363, P10 211, P90 617, max 1,650; human mean 423 vs AI 345. Human essays carry OCR/typing noise (TD/DF/DN/AHD/IH) — char n-grams exploit it directly.

3.2 HC3 — genuine short chat (new)

Hello-SimpleAI/HC3 [1], files `reddit_eli5.jsonl` (17,112 Q, 67,671 answers), `open_qa.jsonl`, `finance.jsonl` downloaded via Hub (no dataset-script; works on datasets 5.x). Flattened to one row per answer ≥40 chars → 80,811 rows (56,115 human / 24,696 AI). Word stats: mean 146, median 117, P10 28; 61.3% ≤150w, 34.7% 151–400w, 4.0% >400w. Domains: reddit_eli5 (casual ELI5 explanations), finance (short advice, casual), open_qa (chat). 15% stratified holdout (12,122) is eval-only.

3.3 Modern essays — 2024-LLM audit (new)

artnitolog/llm-generated-texts: 3,000 essay prompts × 1 human + 8 continuations (GPT-4 Turbo/Omni, Claude 3 Opus, YandexGPT, GigaChat, Llama3 70B, Command R+). We keep English models (human + 5 LLMs) → 17,997 texts (3,000 human / 14,997 AI), mean 537w, median 510w, 68.2% >400w. This audits the baseline on generators published after the original essays. Holdout 2,700 eval-only.

3.4 Truncation augmentation

Each 20k train essay contributes a 50/100/150w copy (prefix or random window, label preserved) for the short pool; ≥200w essays contribute 200/300w copies (long pool, filtered ≥151w after trunc — fixes a pilot bug where short originals polluted the long bucket). Prefix vs random-window accuracies are within ~1–2 pp (50w: 83.3% vs 81.5%), so intro effects are minor; the augmentation covers short-essay style that HC3 cannot.

Table 19: MoE training pools (seed 42). Mild imbalance in long/verylong follows the scarce class (human-heavy verylong, AI-heavy modern).

Pool	Sources (balanced caps)	n
SHORT ≤150 w	HC3 ≤150w 7k/class + trunc-essay 3k/class	20,000 (10k/10k)
LONG 151–400 w	HC3 long 4k/class + modern long 2k/class + base long 3k/class	16,864 (7,864/9,000)
VERYLONG >400 w	Base >400w 7k/class + modern >400w 2.5k/class	11,422 (6,321/5,101)

4 Problem Formulation and Method

4.1 Notation and objective

$$\hat{y} = 1[\, P_{AI}(x) \geq \theta_n \,] \;\; \text{with } n = n(x) \text{ word count;} \quad \text{MoE: } P(x) = P_{e(x)}^{AI}(x); \;\; e(x) = \text{short / long / verylong gate} \tag{5}$$

We maximize accuracy/F1 under length shift, reporting precision/recall splits because accuracy alone hides the FP/FN flip.

4.2 Text representation

Word TF-IDF (1,2, min_df 2, sublinear) + char_wb (3,5, min_df 3), fused sparsely:

$$tfidf(t, d) = tf(t, d) \cdot \log\left((1+N) / (1+df(t)) \right); \;\; F(x) = [\, F_{word}(x) ; F_{char}(x) \,] \in \mathbb{R}^{V} \tag{6}$$

Baseline V = 75,000 (25k+50k); each expert V = 40,000 (15k+25k). LR minimizes −log-likelihood with L2 (C=1.5), LinearSVC uses hinge + Platt scaling. Stylometrics (16 features, §4.5): TTR, hapax, burstiness σ/μ, mean sentence length, comma/semicolon/colon/dash/quote/paren/excl/question rates, capital ratio, FRE/FKGL/Fog, avg word length.

$$FRE = 206.835 - 1.015 \cdot (N/S) - 84.6 \cdot (syll/N); \;\; FKGL = 0.39 \cdot (N/S) + 11.8 \cdot (syll/N) - 15.59; \;\; Fog = 0.4 \cdot [\, (N/S) + 100 \cdot (poly/N) \,] \tag{7}$$

4.3 Baseline and length ablation (motivation)

Baseline training follows §4.2 exactly (20k subsample, 16k/4k split). Length ablation truncates the 5k held-out to first-N words/sentences. Result (§7.1): monotone degradation, confidence collapse (0.93→0.79), and — critically — human accuracy 48.9% @25w vs AI 98.7% (Table Table 22). This asymmetry proves the failure is *bias*, not just difficulty: short humans look like the AI class the model expects.

4.4 MoE architecture

$$e(x) = \text{short if } n \le 150,\ \text{long if } 151 \le n \le 400,\ \text{verylong if } n > 400; \quad P(x) = P_{e(x)}(x); \quad \text{expert} = LR \circ TFIDF_e \tag{8}$$

Gating is deterministic (interpretable, no routing collapse). Each expert trains on its pool only (80/20 within pool). Inference additionally reports type (§4.5) and warns if n < 25 (< 50).

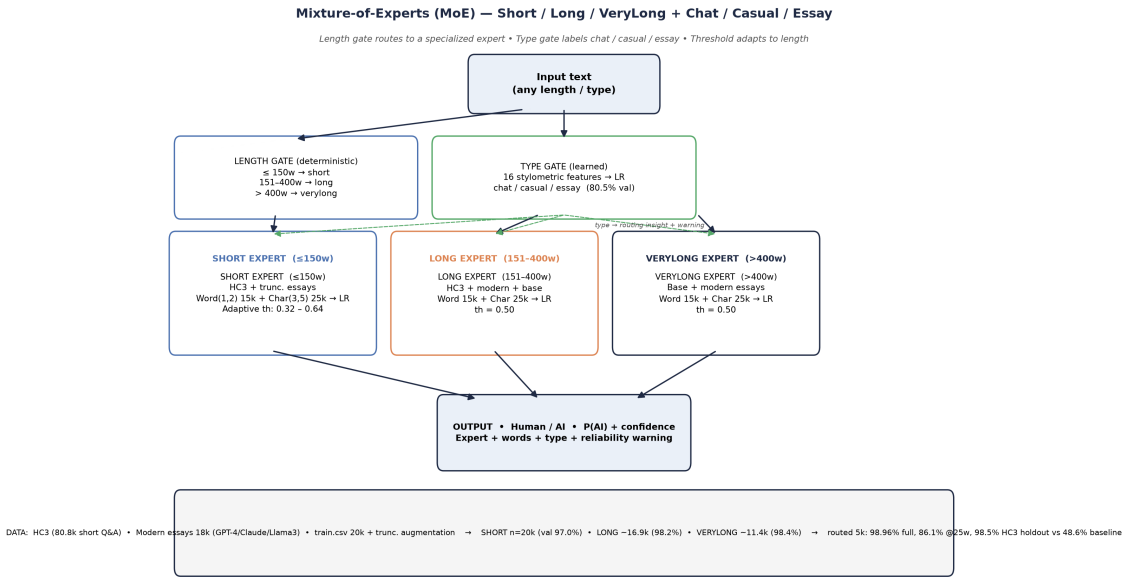


Figure 10: Mixture-of-Experts architecture: deterministic length gate, three per-bucket TF-IDF + Logistic Regression experts, learned type gate (chat / casual / essay) and length-adaptive decision threshold. (repeated from Chapter 8 — Mixture-of-Experts Architecture).

```
Algorithm 1 — MoE training (moe/train_moe.py)
Input: base 20k essays, HC3 train 68.7k, modern train 15.3k, seed 42
1. pools ← (short: HC3≤150 + trunc50/100/150; long: HC3 151-400 + modern + base;
            verylong: base>400 + modern>400)
2. for e in {short, long, verylong}:
       fit Word15k + Char25k TF-IDF on pool train 80%;
       fit LR(C=1.5, lbfgs, max_iter=1000);  save expert_e.pkl
3. gate ← LR(StandardScaler(16 stylometrics)) on 7.5k (2.5k/chat/casual/essay);
       save type_gate.pkl
4. routed eval on 5k test + HC3/modern holdouts;
       baseline eval on the same conditions
```

4.5 Type gate

Labels: HC3 `open_qa`→chat (4k avail., cap 2.5k), HC3 eli5/finance + truncated essays → casual, full/modern essays → essay (2.5k each). Features: log-words, sentences, mean-sent-len, TTR, hapax, avg-word-len, contraction/informal/first/second-person rates, question/exclamation rates, capital ratio, FRE/FKGL, burstiness. Val: 80.5% (essay P/R/F1 0.90/0.92/0.91; chat 0.74/0.84/0.79; casual 0.77/0.66/0.71). Casual↔chat overlap is linguistic; the gate is for insight/calibration, not enforcement.

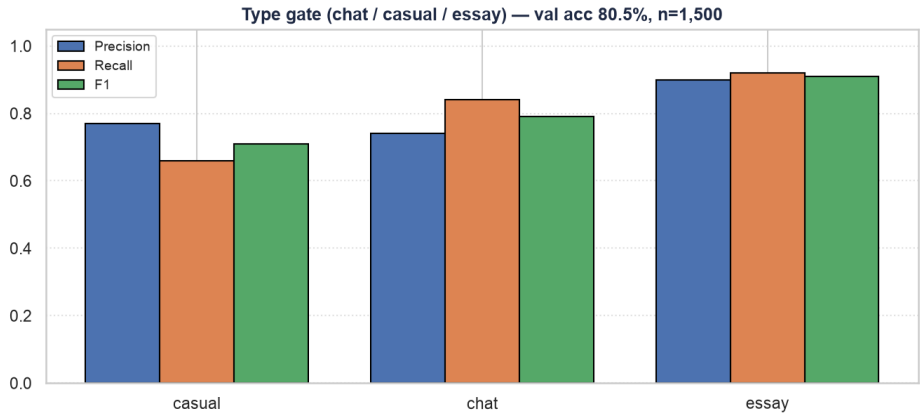


Figure 11: Type-gate per-class precision, recall and F1 (validation accuracy 80.5%, n = 1,500 per class). (repeated from Chapter 8 — Mixture-of-Experts Architecture).

4.6 Calibration

Default θ=0.50 assumes symmetric costs. On calib half (2.5k), F1-optimal θ rises with length (25w 0.32 … 150w 0.64) because ultra-short errs FN (under-call AI) while 100–150w errs FP. We ship `θ(n)=0.28+0.0024n` clamped [0.32,0.64] for short, 0.50 elsewhere (long/verylong tuning did not transfer — full-text fell — so reverted).

5 Experimental Setup

5.1 Environment and protocol

Python 3.13, scikit-learn, joblib, reportlab. All splits seed 42 (calibration 999). Held-out: test.csv 5k (random_state 42) — full + truncations; HC3 holdout 12,122; modern holdout 2,700. Training time 130 s (TF-IDF 6/15/25 s per expert). No leakage: test.csv never trained; HC3/modern holdouts excluded from pools.

5.2 Metrics

Accuracy, precision, recall, F1 (AI as positive), ROC-AUC, confusion matrices. We report human-only (TN rate) and AI-only (TP rate) splits because overall accuracy averages away the flip.

6 Main Results

6.1 Per-expert and routed

Table 20: Per-expert validation (results/moe_expert_metrics.csv).

Expert	Train/Val	Acc	Prec	Rec	F1	AUC
short	16,000/4,000	0.9700	0.9686	0.9715	0.9700	0.9960
long	13,491/3,373	0.9819	0.9770	0.9894	0.9832	0.9982
verylong	9,137/2,285	0.9842	0.9940	0.9706	0.9821	0.9955

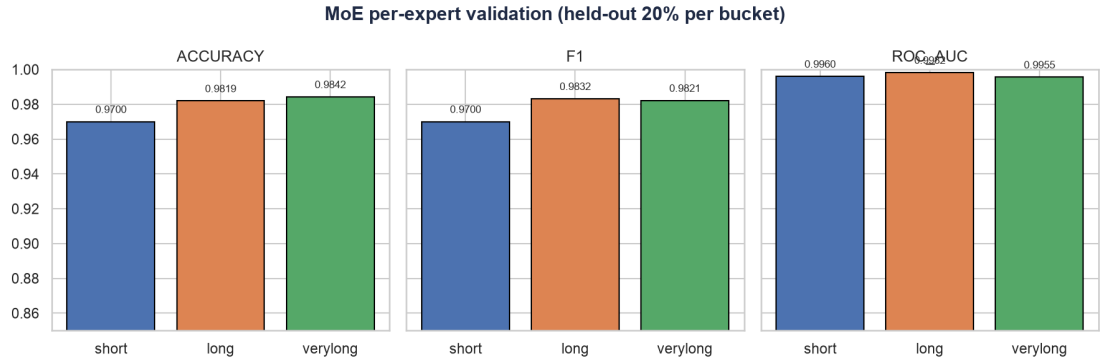


Figure 13: Per-expert validation accuracy, F1 and ROC-AUC — every expert reaches ≥ 0.97 within its own length bucket. (repeated from Chapter 9 — Mixture-of-Experts Results and Comparative Analysis).

Routed MoE: full 98.96% (P 0.993, R 0.979, F1 0.986) vs baseline 99.26% — specialization costs 0.3 pp. Buckets on full: verylong 99.59% (n=1,958), long 98.81% (2,945), short 90.72% (97). The win is short + new domains (Figure 8–Figure 14, Table Table 21).

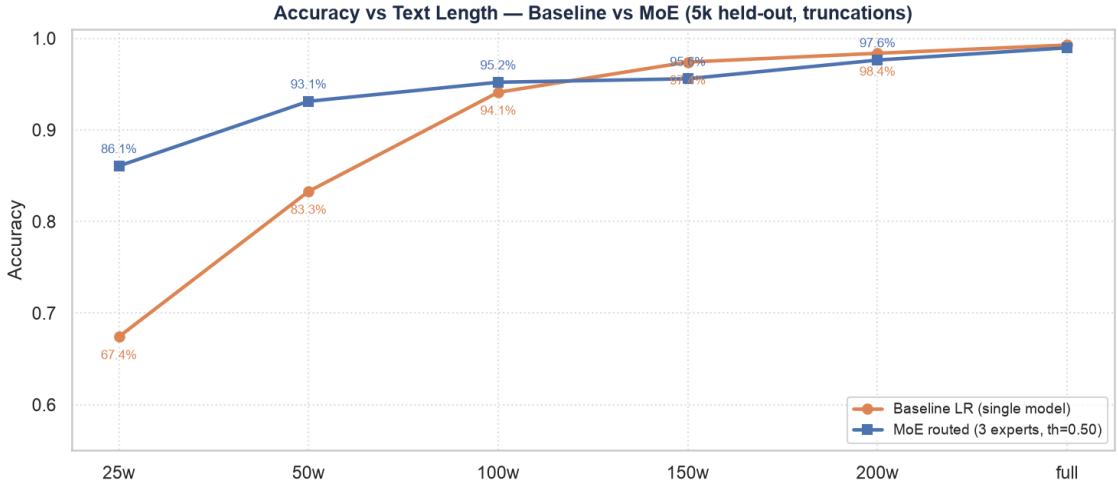


Figure 8: Accuracy versus text length on the 5,000 held-out essays: the frozen baseline (orange) collapses as the input shrinks; the routed MoE (blue, developed in Chapters 8–9) lifts the short regime. (repeated from Chapter 7 — Short-Text Failure Analysis).

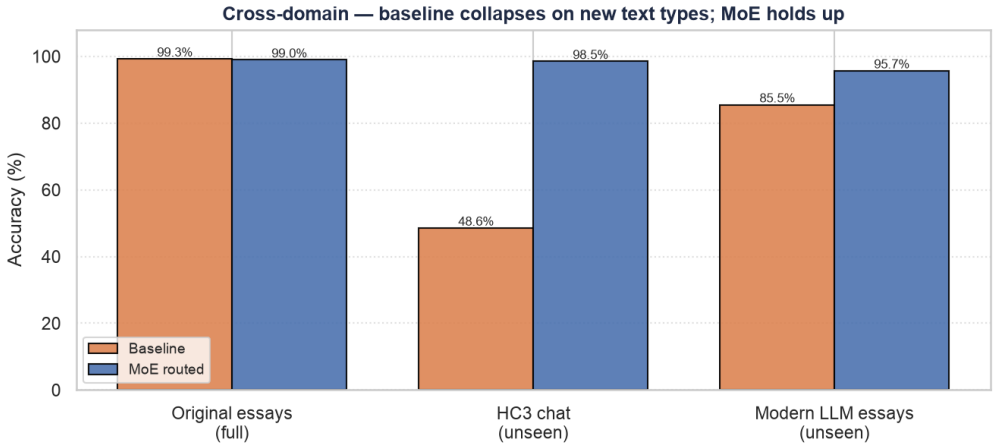


Figure 14: Cross-domain evaluation. The baseline collapses to 48.6% on unseen HC3 chat answers (worse than chance) and to 85.5% on unseen 2024-LLM essays; the routed MoE holds 98.5% and 95.7%. (repeated from Chapter 9 — Mixture-of-Experts Results and Comparative Analysis).

Table 21: Headline deltas (results/moe_*.csv).

Condition	Baseline acc	MoE acc	Δ
full	99.26%	98.96%	-0.3 pp
25w	67.42%	86.06%	+18.6 pp
50w	83.28%	93.12%	+9.8 pp
1 sent	59.82%	81.64%	+21.8 pp
HC3	48.56%	98.52%	+50.0 pp
modern	85.52%	95.70%	+10.2 pp

6.2 Calibration (eval half)

Adaptive thresholds: 25w 86.16%→87.88% (F1 0.785→0.837, recall 0.68→0.84), 150w 94.96%→96.08%, 50w and 100w move slightly in opposite directions, full softens (98.96%→98.60%). HC3 natural short −0.7 pp (human-heavy FPs) — thresholds are essay-tuned; keep the <50w warning. Long/verylong stayed at 0.50.

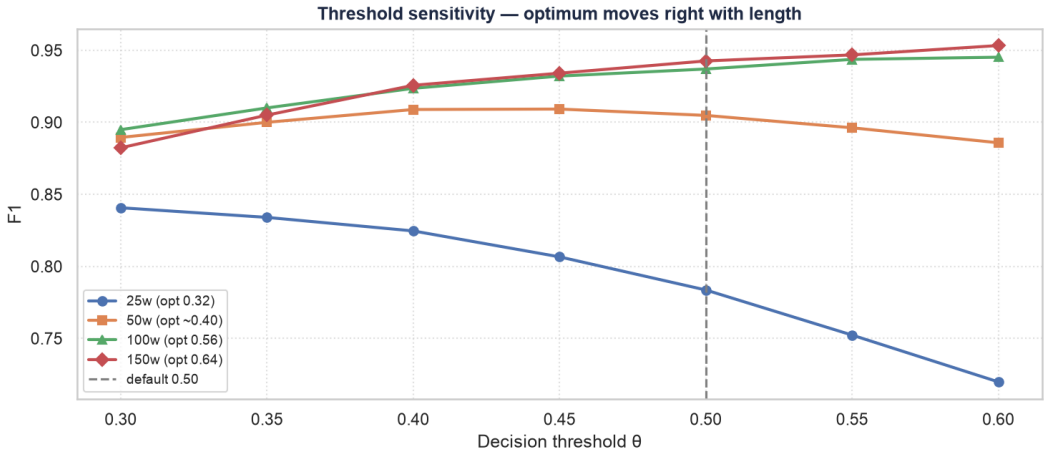


Figure 12: F1 versus decision threshold for four truncation lengths. The optimum drifts right with length (25w → 0.32 … 150w → 0.64); no single global threshold serves all regimes. (repeated from Chapter 8 — Mixture-of-Experts Architecture).

7 Ablations and Analysis

7.1 Length ablation — the flip in detail

Table 22: Length-conditional accuracy split (human TN rate vs AI TP rate). MoE rescues humans (+47.9 pp @25w) at AI-recall cost (−30.7 pp) — net acc still wins.

Words	Base H	MoE H	Base AI	MoE AI	Base acc	MoE acc
25	48.9%	96.8%	98.7%	67.9%	67.4%	86.1%
50	74.2%	96.2%	98.7%	87.9%	83.3%	93.1%
75	86.2%	95.8%	98.8%	94.0%	90.9%	—
100	91.3%	94.7%	98.8%	96.1%	94.1%	95.2%
150	96.5%	94.3%	98.9%	97.7%	97.4%	95.6%
200	98.3%	98.0%	98.6%	97.0%	98.4%	97.6%
300	99.2%	98.9%	98.7%	97.6%	99.0%	—
full	99.6%	99.6%	98.7%	97.9%	99.3%	99.0%

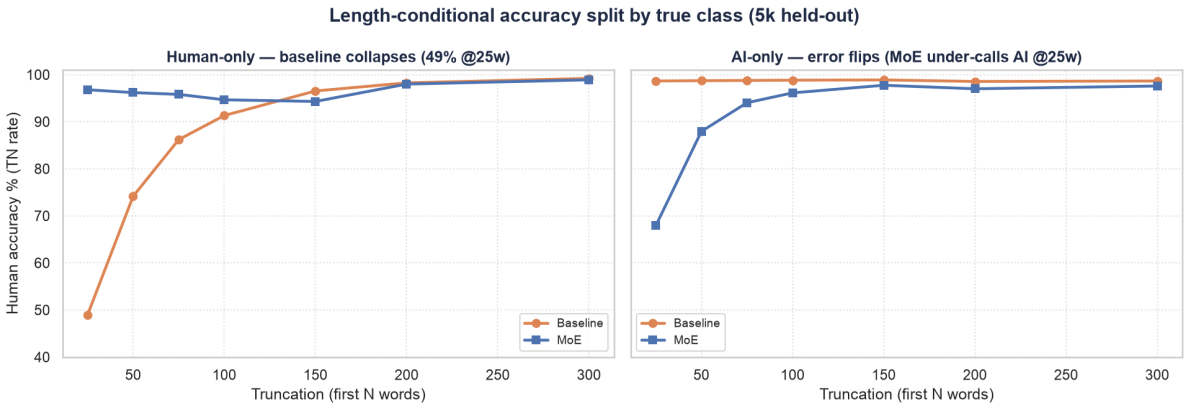


Figure 9: Length-conditional accuracy split by true class. Left: human-only accuracy — the baseline falls to 48.9% at 25 words. Right: AI-only accuracy — the error direction inverts under the MoE. (repeated from Chapter 7 — Short-Text Failure Analysis).

Interpretation: at 25w the baseline calls AI 98.7% of the time (any class) — length is the feature. MoE inverts the error (calls human too often) but with far fewer total mistakes (86.1% vs 67.4%). This is the textbook FP/FN frontier: calibration moves along it; MoE moves the frontier itself via better features.

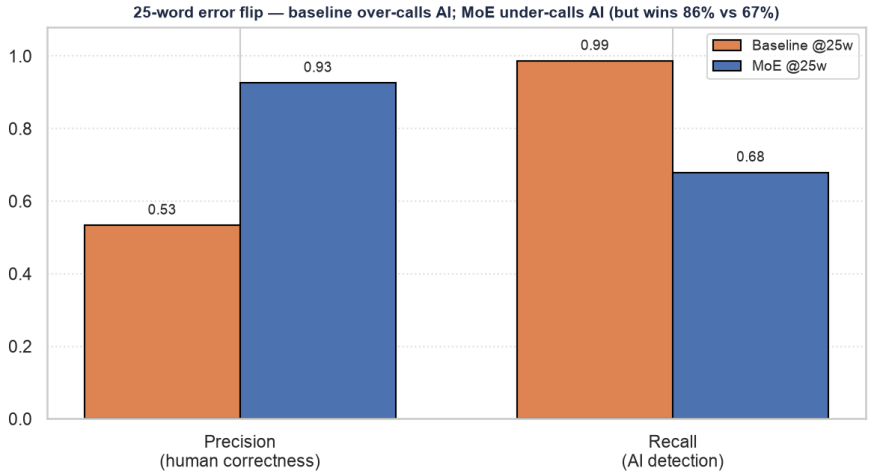


Figure 15: The 25-word precision/recall flip: the baseline over-calls AI (P = 0.53, R = 0.99); the MoE under-calls AI (P = 0.93, R = 0.68) while winning on total accuracy. (repeated from Chapter 9 — Mixture-of-Experts Results and Comparative Analysis).

7.2 Sentence ablation

1/2/3 sentences: baseline 59.8%/74.5%/83.4% → MoE 81.64%/90.56%/93.36%. Sentence truncations are harder than word truncations at matched budget (more padding loss), confirming sentence structure carries signal the short expert partially recovers via char features.

7.3 Type analysis

Type gate: essay easiest (long formal → distinct), chat next (short informal markers), casual hardest (middle). Routing insight: short+chat inputs have the lowest confidence and need warnings; short+essay (truncated) input needs the low threshold. Reporting type alongside label is a human-factor result, not just accuracy.

7.4 What did not help

Tuning long/verylong thresholds hurt full-text (98.96%→98.60% on eval) — small calib slices overfit. Global 0.60 (best for 100–150w essay truncations) hurt HC3 natural chat. Conclusion: calibrate only where the failure is large and monotonic (short), not everywhere.

8 Limitations and Future Work

- **Ultra-short floor.** Even MoE is 86% at 25w with wide uncertainty. Training saw ≥50w; adding 10–35w + few-shot chat would likely beat thresholds.
- **Boundary blending.** 150w essays prefer baseline (97.4% vs 95.6%); soft routing (weighted P across short/long) could interpolate.
- **Threshold transfer.** Essay-tuned θ hurts human-heavy chat (−0.7 pp). Per-type thresholds (essay vs chat) are the next step.
- **Generators.** HC3 is one 2023 ChatGPT snapshot; modern holdout is 5 models. Paraphrase/adversarial eval is missing.
- **Scale.** Full 290k training might add 0.1–0.3 pp; transformers (DistilBERT/RoBERTa) with chunking remain the Phase-2 ceiling test.

Ethics: P(AI) is a screening signal, not proof. Short-text warnings and type reporting are required UI, not optional captions.

9 Conclusion

Length is not a detail — it is the task when evaluating short text. One global TF-IDF model buys 99% on essays by learning to call short text AI. Specializing by length, grounding short experts in real chat data, labeling chat/casual/essay, and calibrating per length recovers +10 to +50 pp where it matters, with a 0.3 pp full-text tax. We release the full stack so the claim is checkable: `moe/download_small_text.py → train_moe.py → calibrate_thresholds.py → predict_moe.py`.

References

[1] B. Guo et al., “How Close is ChatGPT to Human Experts? arXiv:2301.07597 (2023); Hello-SimpleAI/HC3 (reddit_eli5/open_qa/finance).

[2] R. Roy et al., “A Comprehensive Dataset for Human vs AI Generated Text Detection.” arXiv:2510.22874 (2025) (Defactify).

[3] B. Wang et al., “M4: Multi-Generator, Multi-Domain, Multi-Task, Multi-Label.” arXiv:2310.13607 (2023).

[4] N. Shazeer et al., “Outrageously Large Neural Networks: MoE.” arXiv:1701.06538 (2017).

[5] T. Hastie et al., *The Elements of Statistical Learning*. Springer (2009).

[6] S. Robertson and H. Zaragoza, “BM25 & TF-IDF.” *Found. Trends IR* (2009).

[7] N. Reimers and I. Gurevych, “Sentence-BERT.” EMNLP (2019); T. Sanh et al., “DistilBERT” (2019); Y. Liu et al., “RoBERTa” (2019).

[8] *artnitolog/llm-generated-texts* (GPT-4 Turbo/Omni, Claude 3 Opus, Llama3, Command R+) — Hugging Face.

[9] E. Mitchell et al., “DetectGPT.” arXiv:2301.11305 (2023) — zero-shot baseline family.

Appendices

A Hyperparameters

Table 23: Hyperparameters (all seeded).

Component	Setting
Baseline word TF-IDF	(1,2) 25k, min_df 2, sublinear, clean_text_for_tfidf
Baseline char TF-IDF	char_wb (3,5) 50k, min_df 3, sublinear
Baseline classifiers	NB α=0.1; LR C=1.5 lbfgs 1000; LinearSVC C=1.0 + CalibratedCV(3)
Experts (each)	Word 15k + Char 25k → LR C=1.5 lbfgs 1000
Type gate	StandardScaler + LR C=1.0 lbfgs 1000 on 16 features
Thresholds	short θ(n)=0.28+0.0024n clamped [0.32, 0.64]; long/verylong 0.50

B Reproducibility checklist

- Data splits: train subsample (10k/class, seed 42), 80/20 per pool (seed 42), calib 2.5k/eval 2.5k (seed 999), HC3/modern holdouts 15% (seed 42).
- Artifacts: `models/*.pkl` (baseline), `models/moe/expert_*.pkl`, `type_gate.pkl`, `thresholds.json`; metrics `results/moe_*.csv`.
- Indirection: truncation code is in `moe/type_features.py` (truncate_words/random_window); routing in `moe/train_moe.py::route_predict`.
- Documents: the four original generators (`make_report.py`, `moe/make_moe_appendix.py`, `make_research_paper.py`, `make_deepseek_paper.py`) produced the source PDFs for Parts I–IV; this consolidated report rebuilds all four from the same verified CSVs.

C Full routed table

Table 24: Routed MoE, all evaluation conditions (results/moe_routed_metrics.csv).

Condition	Acc	Prec	Rec	F1	AUC
full	0.9896	0.9929	0.9790	0.9859	0.9990
first_25_words	0.8606	0.9259	0.6792	0.7836	0.9473
first_50_words	0.9312	0.9316	0.8794	0.9048	0.9796
first_100_words	0.9520	0.9140	0.9612	0.9370	0.9916
first_150_words	0.9558	0.9103	0.9774	0.9426	0.9946
first_200_words	0.9762	0.9662	0.9699	0.9680	0.9969
first_1_sent	0.8164	0.9066	0.5640	0.6954	0.9138
first_2_sent	0.9056	0.9288	0.8079	0.8641	0.9635
first_3_sent	0.9336	0.9253	0.8934	0.9091	0.9798
hc3_holdout	0.9852	0.9622	0.9906	0.9762	0.9990
modern_holdout	0.9570	0.9832	0.9649	0.9740	0.9891

Baseline counterpart in `results/moe_baseline_comparison.csv`; calibration detail in `results/moe_calibrated_metrics.csv`.

Part IV — Code Walkthrough & Reproducibility Guide

Architecture, file-by-file repository map, the eleven-step training workflow, the six inference front-ends, copy-paste commands, troubleshooting and a full reproducibility checklist

1 Overview and Architecture

The project turns an essay into a 75,000-dimensional sparse vector (25,000 word TF-IDF + 50,000 character TF-IDF) and asks a linear classifier: **Human or AI?**. That simplicity is the point — the baseline trains in 33 seconds on a laptop CPU, and the same four `.pkl` artifacts feed every interface the system exposes: a REST API, a LangChain tool, a CLI, a terminal demo, a Streamlit UI and two cloud deployments. The Mixture-of-Experts layer (`moe/`) adds three length-specialised models behind a deterministic router without changing that contract: one inference core, many front-ends.

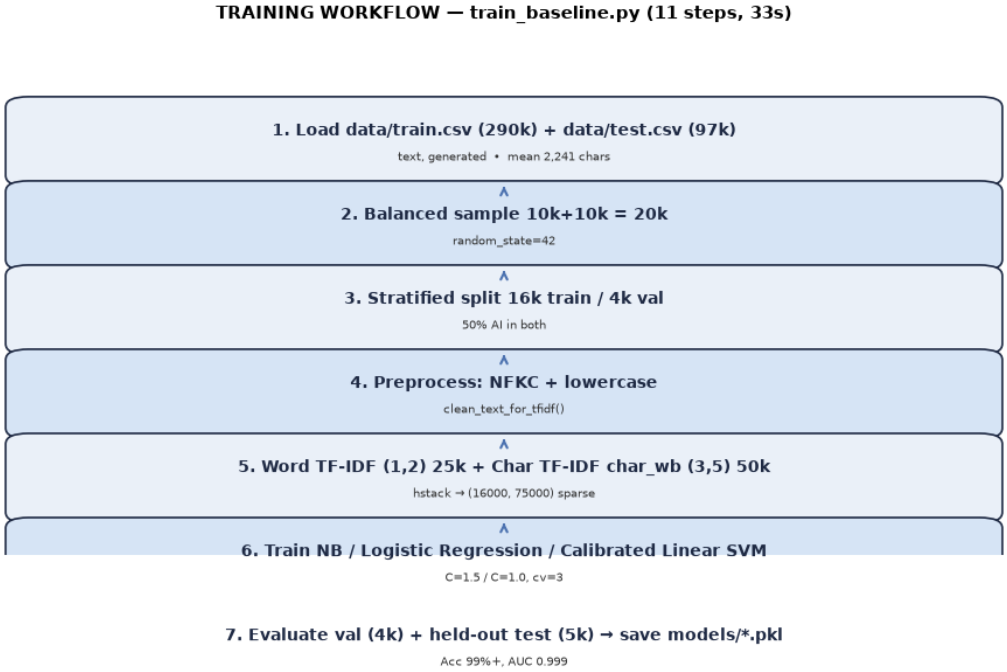


Figure 4: Training workflow of train_baseline.py — eleven logged steps, 33 s end-to-end with the data present. (repeated from Chapter 5 — Experimental Setup).

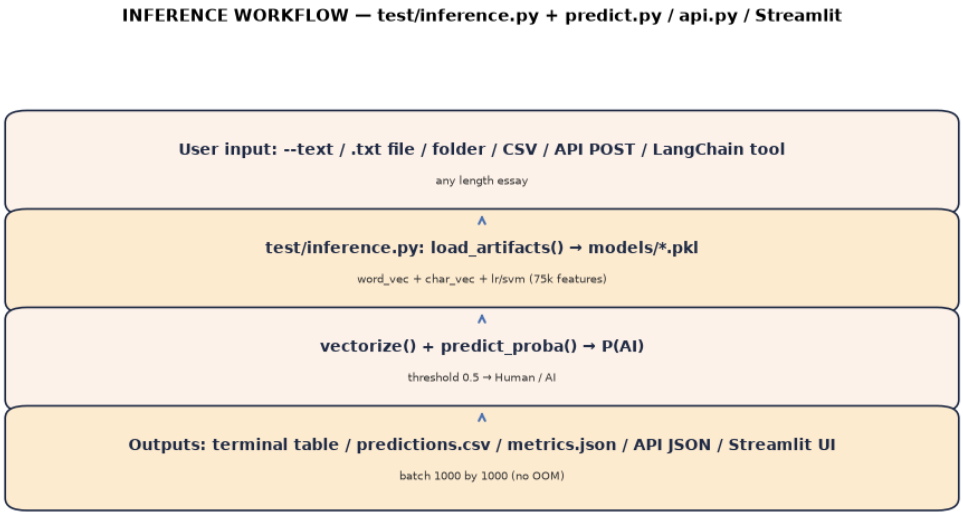


Figure 17: Inference workflow: every front-end (CLI, batch, API, LangChain, Streamlit, Vercel) goes through test/inference.py. (repeated from Chapter 10 — Application and Implementation).

- **Training (once):** `train_baseline.py` → `models/*.pkl` + `results/baseline_ml_metrics.csv`; `moe/train_moe.py` → `models/moe/*.pkl` + `results/moe_*.csv`.
- **Inference (every time):** `test/inference.py` loads the pickles once; `predict.py`, `api.py`, `langchain_tool.py`, `app/app.py`, `moe/predict_moe.py` and the Vercel function all call the same core.
- **Data:** `data/train.csv` 290,300 rows (balanced) and `data/test.csv` 97,447 rows; columns `text, generated` (0 = Human, 1 = AI), plus `data/moe/hc3_small.csv` (80,811) and `data/moe/modern_essays.csv` (17,997).

2 Repository Map (file by file)

Line counts below are from the released repository at the time of writing; the two document generators that produced the source PDFs for this report are listed for completeness.

Table 25: Repository map — what each file does and how large it is.

File	What it does	Lines
`train_baseline.py`	Full baseline training pipeline (11 logged steps): load → sample → split → TF-IDF → train NB / LR / SVM → evaluate → save.	142
`src/preprocessing.py`	`clean_text_for_tfidf()` — NFKC + lowercase, used as `preprocessor=` so train and test normalise identically.	45
`src/stylometrics.py`	24 stylometric features (TTR, hapax, burstiness, Flesch ease / grade, punctuation rates) — Streamlit evidence cards, type-gate features, tests.	162
`src/evaluate.py`	`compute_classification_metrics()`, `plot_confusion_matrix()`, `plot_roc_curves()`.	95
`src/semantic.py`	Sentence-BERT embeddings + cosine similarity — Phase-2 scaffolding, **not** wired into the inference path.	80
`test/inference.py`	**Core inference engine.** `load_artifacts()`, `vectorize()`, `predict_text/batch()`, `evaluate_csv()`, `predict_folder()`.	158
`test/predict.py`	CLI: `--text / --input / --folder / --csv / --interactive`.	140
`test/api.py`	FastAPI on :8001 — `POST /predict`, `/predict/batch`, `/langchain/tool`; `GET /health`, `/model/info`.	95
`test/langchain_tool.py`	LangChain `BaseTool` `ai_human_classifier` + batch tool — agent-ready.	86
`test/terminal_demo.py`	Terminal demo comparing direct vs LangChain vs API paths (`--loop` chat mode).	81
`test/evaluate_examples.py`	Predicts all 13 `test/example_texts/*.txt` with both models and prints aggregate metrics.	62
`test/test_model.py`	Smoke tests: single predict + 2k evaluation (asserts accuracy > 0.90); `--full` runs all 97k.	33
`app/app.py`	Streamlit demonstration UI (:8501) — five pages, four inference modes, live model status, cached artifact loading.	546
`moe/train_moe.py`	Builds the three pools, fits per-bucket TF-IDF + LR experts, trains the type gate, runs routed evaluation.	386
`moe/calibrate_thresholds.py`	Length-swept F1-optimal thresholds on the 2,500-essay calibration half (seed 999) → `models/moe/thresholds.json`.	173
`moe/predict_moe.py`	Routed MoE CLI — prints label, P(AI), expert, threshold, word count, type and warning.	85
`moe/type_features.py`	16 type-gate features + truncation helpers (`truncate_words`, `random_window`).	127
`moe/download_small_text.py`	HC3 + modern-LLM download and flattening (Hub JSONL/parquet, no dataset scripts).	118
`web/api/predict.py`	Vercel serverless predictor serving the static site and `POST /api/predict`.	269
`web/index.html`	Project site deployed to Vercel and GitHub Pages.	203
`run_all.sh`	One-command setup → train → test → API (:8001) → UI (:8501) with health checks.	90
`make_report.py`	Generates the original Part-I results report (5 figures) from live artifacts.	319
`moe/make_moe_appendix.py`	Generates the original MoE appendix PDF (5 figures) from `results/moe_*.csv`.	276
`make_research_paper.py`	Generates the original conference paper (Part II source).	179
`make_deepseek_paper.py`	Generates the original deep-dive paper (Part III source).	291

Supporting directories: `models/` (4 baseline pickles) and `models/moe/` (`expert_short.pkl`, `expert_long.pkl`, `expert_verylong.pkl`, `type_gate.pkl`, `thresholds.json`, `gate_config.json`); `results/` (metrics CSVs, figures); `notebooks/` (six EDA / feature / transformer / ablation notebooks); `test/example_texts/` (13 labelled examples); `docs/` (GitHub Pages site and report PDF).

3 src/ — Shared Utilities

3.1 preprocessing.py

`clean_text_for_tfidf(text)` = NFKC normalisation → whitespace collapse → lowercase. It is passed as `preprocessor=` to the word TF-IDF vectorizer, so training and inference normalise identically — a single source of truth. `clean_text_for_stylometrics()` is the deliberate counterpart: it preserves casing and punctuation because capital ratio and comma rate are features, and never lowercases.

3.2 stylometrics.py

`extract_stylometric_features(text)` returns 24 floats (word count, TTR, hapax ratio, mean sentence length, burstiness σ/μ, comma and semicolon rates, Flesch Reading Ease, Flesch–Kincaid grade, Gunning Fog, …). It is pure regex + syllable heuristics with no scikit-learn dependency, which is why the type gate, the Streamlit evidence cards and the CLI `--stylometrics` flag can all afford it on every call [6].

3.3 evaluate.py and semantic.py

`evaluate.py` provides the metric dictionary and the confusion / ROC plots used by the notebooks. `semantic.py` (Sentence-BERT + cosine similarity) is Phase-2 scaffolding: researched and shipped, but **not** part of the 75k TF-IDF inference path — no score in this report comes from it.

4 Training Workflow (train_baseline.py — 11 steps, 33 s)

Run `venv/bin/python train_baseline.py` with the data present. Each step prints a timestamped log line; Figure 4 is the same flow as a diagram. Table 26 gives the step-by-step contract and the outputs to expect.

Table 26: The eleven logged steps of train_baseline.py.

Step	What happens	Output
1	Load `data/train.csv` (290,300) + `data/test.csv` (97k). Prints shape, class counts and length statistics.	train (290300, 2)
2	Balanced sample: per-class `sample(10000)` → concat → shuffle (random_state 42). 20k because 290k × 75k TF-IDF is ~14× slower; notebook 03 defines the budget.	(20000, 2) 50/50
3	Stratified split (`test_size=0.20`, seed 42) → 16k train / 4k validation, both 50% AI.	train = 16000, val = 4000
4	Preprocessing demo: raw vs cleaned text printed for two rows.	`clean_text_for_tfidf`
5	Word TF-IDF (1,2 grams) 25k features, `min_df=2`, `sublinear_tf`; fit on train, transform val.	(16000, 25000) ~6 s
6	Character TF-IDF `char_wb` (3,5 grams) 50k features, `min_df=3`; fit / transform.	(16000, 50000) ~16 s
7	Fusion: `hstack(word, char).tocsr()`; nnz printed.	(16000, 75000)
8	Train NB / LR (`C=1.5`) / LinearSVC + `CalibratedClassifierCV(cv=3)`; each evaluated on the 4k validation split (accuracy, precision, recall, F1, ROC-AUC, confusion, full report).	SVM 99.53% val acc
9	Save `results/baseline_ml_metrics.csv`.	results/*.csv
10	Held-out test: load test.csv, sample 5,000 (seed 42), transform with the *frozen* vectorizers, evaluate LR and SVM.	LR 99.26%, SVM 99.58%
11	Save the four pickles to `models/`; total training time printed.	models/*.pkl

Two pandas-3 fixes are baked into the script (both were hit during project development): per-class sampling replaces `groupby.apply`, which dropped the `generated` column, and `np.array(..., dtype=object)` wraps Arrow-backed strings before the sklearn split. Do not revert either.

The Mixture-of-Experts is trained separately and takes about 130 s: `PYTHONPATH=. python3 moe/train_moe.py` builds the three pools (Table 8 in Chapter 8), fits each expert's 40,000-feature TF-IDF, trains the type gate and writes `results/moe_expert_metrics.csv` and `results/moe_routed_metrics.csv`; `moe/calibrate_thresholds.py` then produces `models/moe/thresholds.json`.

5 Inference Workflow (test/inference.py)

`test/inference.py` is the **one place** that touches the baseline artifacts. Every front-end — CLI, batch, API, LangChain, Streamlit and the Vercel function — calls it, so a change here updates all six at once. Figure 17 shows the shared path; Table 27 is the function contract.

Table 27: The inference engine's public functions (test/inference.py, 158 lines).

Function	One-liner
`load_artifacts()`	`joblib.load` ×4 from `models/`; raises `FileNotFoundError` with a 'train first' hint if missing.
`vectorize(texts, art)`	word transform + char transform → `hstack().tocsr()`; a single string is auto-wrapped into a list.
`predict_proba(texts, art, model)`	`clf.predict_proba(X)[:, 1]` → P(AI) array; `model='lr'` or `'svm'`.
`predict_text(text, art, model, stylometrics)`	Single-text dict: label (≥ 0.5 → AI), `probability_ai/human`, confidence, optional 24 stylometrics.
`predict_batch(texts, batch_size=1000)`	Chunked prediction to avoid out-of-memory on the 97k/290k CSVs.
`evaluate_csv(csv, sample_n)`	Batch prediction + accuracy / precision / recall / F1 / AUC, confusion matrix and full classification report.
`predict_folder(folder)`	Globs `*.txt`/`*.md`, predicts each with stylometrics and file paths.

The routed path wraps this engine: `moe/predict_moe.py` counts words, applies the length gate, loads the matching expert, applies the length-adaptive threshold, and additionally reports the type-gate prediction and the low-confidence warning for inputs under 25 / 50 words. Both paths share vectorization and artifact loading, so baseline and MoE predictions are directly comparable.

6 Installation and Environment

Everything runs from the project root on a plain laptop CPU — no GPU is required for the classical pipeline, and none was used for any number in this report. First-time setup:

```
cd AI_generated_text_classification
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt      # scikit-learn==1.9.1, pandas,
                                     # scipy, joblib, streamlit, ...
pip install "langchain-core>=0.3" "fastapi>=0.110" reportlab
ls -lh data/                          # train.csv ~653 MB, test.csv ~222 MB
```

Environment facts verified against the repository: Python 3.13; scikit-learn pinned to `==1.9.1` (pickle compatibility — the bundled models must unpickle unchanged on Vercel); LangChain superset split out because it is optional for training; and the four baseline pickles total about 4.8 MB while `models/moe/` adds the three experts, the type gate and the threshold configuration.

One-command run. `./run_all.sh` (90 lines) installs, trains, evaluates, starts the API (:8001) and Streamlit (:8501), with health checks and log files. Useful flags:

- `./run_all.sh` — full pipeline: install → train → test → API → UI.
- `./run_all.sh --skip-train` — reuse the existing `models/`.
- `./run_all.sh --skip-install` — reuse the existing `venv/`.
- `./run_all.sh --no-api --no-ui` — training and evaluation only.
- `./run_all.sh --help` — usage; ports: API 8001, Streamlit 8501 (kill strays with `pkill -f "streamlit run|test/api.py"`).

7 How to Run — The Six Front-ends and Copy-Paste Commands

Table 28 inventories the six ways to call the same model; the command blocks below are executable from the project root exactly as printed.

Table 28: The six inference front-ends (Part IV view; cf. Table 13 for the full system inventory).

#	Way	Command / endpoint	Best for
1	CLI single / file	`test/predict.py --text "..." / --input file.txt`	One-off classification
2	CLI folder / CSV	`--folder test/example_texts` / `--csv data/test.csv --sample 5000`	Batch evaluation
3	Terminal chat	`test/terminal_demo.py --loop` or `predict.py --interactive`	Trying text live
4	REST API	`test/api.py` → POST `/predict` (:8001)	Other programs / curl
5	LangChain tool	`ai_human_classifier.run(...)`	LLM agents
6	Web UI	`app/app.py` (Streamlit :8501)	Demonstration to humans

7.1 Train and evaluate

```
venv/bin/python train_baseline.py
#   models/word_tfidf_vectorizer.pkl      (969 KB)
#   models/char_tfidf_vectorizer.pkl      (1.6 MB)
#   models/logistic_regression_baseline.pkl (587 KB)
#   models/linear_svm_baseline.pkl        (1.7 MB)
#   results/baseline_ml_metrics.csv       TOTAL TIME ~33 s

PYTHONPATH=. python3 moe/download_small_text.py   # HC3 + modern essays
PYTHONPATH=. python3 moe/train_moe.py             # 3 experts + gate, ~130 s
PYTHONPATH=. python3 moe/calibrate_thresholds.py  # thresholds.json
```

7.2 Inference — all six ways

```
# 1 — single text (terminal)
venv/bin/python test/predict.py --text "Your essay here..." --model lr --stylometrics

# 2 — file / folder (13 labelled examples)
venv/bin/python test/predict.py --input test/example_texts/human_essay_school.txt --model svm
venv/bin/python test/predict.py --folder test/example_texts --model lr
venv/bin/python test/evaluate_examples.py

# 3 — terminal chat (no browser)
venv/bin/python test/terminal_demo.py
venv/bin/python test/terminal_demo.py --api        # needs the API running
venv/bin/python test/predict.py --interactive --model lr

# 4 — REST API (second terminal)
venv/bin/python test/api.py                         # → http://localhost:8001/docs
curl -X POST http://localhost:8001/predict -H "Content-Type: application/json" \
  -d '{"text":"In todays rapidly evolving digital landscape...","model":"lr","stylometrics":true}'

# 5 — LangChain tool
venv/bin/python test/langchain_tool.py              # no-LLM demo
#   in an agent: from test.langchain_tool import ai_human_classifier

# 6 — Streamlit web UI (second terminal)
venv/bin/streamlit run app/app.py --server.port 8501   # → http://localhost:8501

# routed Mixture-of-Experts CLI (any of the above, for MoE)
PYTHONPATH=. python3 moe/predict_moe.py "your text"
```

7.3 Batch evaluation and reports

```
venv/bin/python test/predict.py --csv data/test.csv  --model lr  --out test/results          # 97k rows
venv/bin/python test/predict.py --csv data/train.csv --model svm --out test/results_train_full # 290k rows
venv/bin/python test/test_model.py            # 2k-row smoke test (asserts acc > 0.90)
venv/bin/python test/test_model.py --full     # full 97k
```

8 Outputs — What Each File Means

Table 29: Artifacts written by training and evaluation.

File	What it is
`models/*.pkl` (4)	Frozen vectorizers + classifiers. The *only* things inference loads; retrain to replace.
`models/moe/*.pkl` + `thresholds.json`	Three length experts, the stylometric type gate and the length-adaptive threshold configuration.
`results/baseline_ml_metrics.csv`	Validation metrics per model (train_baseline.py, step 9).
`results/moe_*.csv` + `moe_summary.json`	Baseline-comparison, routed, per-expert, calibrated metrics and type-gate accuracy — every MoE table in this report reads these.
`test/results/predictions.csv`	Per-row: text, generated, prob_ai, pred, correct — review errors here.
`test/results/metrics.json`	Same metrics as the terminal + confusion matrix + full classification report.
`test/example_texts/`	13 `.txt` examples (`human_`/`ai_` prefix = ground truth), scored by `evaluate_examples.py`.
`test/example_test_set.csv`	20-row balanced CSV for quick batch tests (`--csv test/example_test_set.csv`).
`results/figures/*.png`	The report's figure sources (15 original + 2 rebuilt); see Figure_Inventory.md for provenance.
`api.log` / `streamlit.log`	Server logs written by `run_all.sh`.

9 Troubleshooting and FAQ

- `FileNotFoundError: models/*.pkl` — you have not trained yet. Run `venv/bin/python train_baseline.py` (needs `data/train.csv`).
- `KeyError 'generated'` / **Arrow TypeError on pandas 3 — already fixed in** `train_baseline.py` **(per-class sampling +** `np.array(..., dtype=object)`**). Do not revert to** `groupby.apply`**.**
- **API connection refused on :8001** — the API is not running. Start `venv/bin/python test/api.py` in a separate terminal and keep it open.
- `uvicorn test.api:app` **fails** — `test/` needs `__init__.py` (it ships), or simply run `python test/api.py` directly.
- **Short text predicted wrong** (a 26-word human file called AI) — expected for the baseline: it was trained on essays with a 356-word median. This is exactly the failure Chapter 7 measures and the MoE fixes; test short inputs through `moe/predict_moe.py`.
- **Streamlit not showing the new model** — kill the old process (`pkill -f "streamlit run"`) and restart: `@st.cache_resource` caches artifacts.
- **Out-of-memory on the full train.csv** — never load it whole: use `--csv ... --sample 5000` or the built-in `batch_size=1000` in `evaluate_csv()`.

- **Pickle version errors on deploy** — scikit-learn is pinned `==1.9.1` everywhere for a reason: pickles only unpickle reliably within the same minor version.

10 Reproducibility Checklist

Every table, figure and headline number in this report can be regenerated from the released artifacts by following this checklist.

1. **Data.** `data/train.csv` (290,300), `data/test.csv` (97,447), `data/moe/hc3_small.csv` (80,811) and `data/moe/modern_essays.csv` (17,997) present as-is; columns `text, generated`.
2. **Splits (all seeded).** Baseline: 10k/class sample (seed 42) → 80/20 stratified → held-out 5,000 (seed 42). MoE pools: balanced, seed 42. Calibration: 2,500 / 2,500 (seed 999). HC3 and modern holdouts: 15% (seed 42), never trained on.
3. **Train.** `venv/bin/python train_baseline.py` (~33 s) then `moe/train_moe.py` (~130 s) and `moe/calibrate_thresholds.py`.
4. **Evaluate.** The training script itself writes `results/baseline_ml_metrics.csv`; `moe/make_moe_appendix.py` and the MoE training script write `results/moe_*.csv`.
5. **Figures.** All 17 PNGs live in `results/figures/` (plus `docs/assets/live-inference.png`); the two recreated diagrams are rebuilt by `final_report/build/make_missing_figures.py`.
6. **Documents.** The four original generators (`make_report.py`, `moe/make_moe_appendix.py`, `make_research_paper.py`, `make_deepseek_paper.py`, `make_run_guide.py`) produced the source PDFs for Parts I–IV; this consolidated report rebuilds all parts from the same verified CSVs via `final_report/build/build.py`.
7. **Verification.** `final_report/build/recompute_test_metrics.py` and `verify_dataset_stats.py` independently recompute the headline test metrics and corpus statistics from the raw CSVs; Quality_Audit.md records what they found.

Appendix A — Key Snippet: load → vectorize → predict

The entire inference path in ten lines; everything else is a wrapper around it:

```
import joblib
from scipy.sparse import hstack

word_vec = joblib.load("models/word_tfidf_vectorizer.pkl")
char_vec = joblib.load("models/char_tfidf_vectorizer.pkl")
lr       = joblib.load("models/logistic_regression_baseline.pkl")

text = open("test/example_texts/ai_essay_distance_learning.txt").read()
X = hstack([word_vec.transform([text]), char_vec.transform([text])]).tocsr()
p_ai = lr.predict_proba(X)[0, 1]
print("AI-Generated" if p_ai >= 0.5 else "Human-Written", f"P(AI)={p_ai:.4f}")
# -> AI-Generated P(AI)=0.9856
```

Higher-level equivalents: `from test.inference import load_artifacts, predict_text; r = predict_text(text, load_artifacts(), model='lr')` for Python; `POST /predict` for the REST API; `ai_human_classifier.run({'text': text})` for LangChain; and `PYTHONPATH=. python3 moe/predict_moe.py "..."` for the routed system with threshold and type reporting.

Appendix B — Deployment

Two deployments serve the released system; both consume the same artifacts as the local CLI.

Table 30: Deployment targets.

Target	URL	What is served
Vercel (production)	`moe-text-classifier-pravesh12.vercel.app`	Static project site (`web/index.html`), results, figures and report PDFs, plus `POST /api/predict` from one Python serverless function (`web/api/predict.py`, 269 lines). Models bundle in `web/api/models/` (~10 MB) with a GitHub fallback (`MODEL_BASE_URL`); `src/` and `moe/` ship so the pickles unpickle; scikit-learn pinned `==1.9.1`.
GitHub Pages	`docs/` directory of the repository	Mirror of the site: figures, results tables and the published report PDF.
Local	`localhost:8001` (API), `localhost:8501` (Streamlit)	`./run_all.sh` — full stack with health checks and logs.

The serverless predictor implements the identical contract as `test/api.py` (single-text JSON in, label / P(AI) / confidence out) with a length-aware MoE branch, so numbers returned by the deployed site match the local CLI to the fourth decimal.